AY 2025/2026



Politecnico di Milano

RASD: Requirement Analysis and Specification Document

Filippo Maretti Davide Martini

Professor
Matteo Giovanni Rossi

Contents

1 Introduction **1**
- A Purpose . . . 1
  - A.1 Goals . . . 1
- B Scope . . . 3
  - B.1 World Phenomena . . . 3
  - B.2 Shared Phenomena . . . 4
- C Definitions, Acronyms, Abbreviations . . . 6
  - C.1 Definitions . . . 6
  - C.2 Acronyms . . . 7
  - C.3 Abbreviations . . . 7
- D Revision History . . . 8
- E Reference Documents . . . 8
- F Document Structure . . . 8

2 Overall Description **9**
- A Product perspective . . . 9
  - A.1 Scenarios . . . 9
  - A.2 Class Diagram . . . 12
  - A.3 State Diagrams . . . 13
- B Product functions . . . 16
  - B.1 User Registration (Sign Up) . . . 16
  - B.2 User Login . . . 17
  - B.3 Record Personal Trip . . . 18
  - B.4 Trip Statistics & Weather Enrichment . . . 19
  - B.5 Automated Data Collection . . . 19
  - B.6 Verify & Confirm Detected Data . . . 20
  - B.7 Manual Path Reporting . . . 20
  - B.8 Publish Path Information . . . 21
  - B.9 Route Planning & Scoring . . . 21
  - B.10 View Statistics & History . . . 22
- C User characteristics . . . 23
- D Assumptions, dependencies and constraints . . . 24
  - D.1 Domain Assumptions . . . 24
  - D.2 Dependencies . . . 24
  - D.3 Constraints . . . 25

3 Specific Requirements **26**
- A External Interface Requirements . . . 26
  - A.1 User Interfaces . . . 26
  - A.2 Hardware Interfaces . . . 26
  - A.3 Software Interfaces . . . 27
  - A.4 Communication Interfaces . . . 27
- B Functional Requirements . . . 28

B.1 Use case diagrams 28
B.2 Use Case Tables and Sequence Diagrams 29
B.3 Requirements 47
B.4 Traceability Matrix 50
C Performance Requirements 50
D Design Constraints 51
D.1 Standards compliance 51
D.2 Hardware limitations 51
D.3 Any other constraint 51
E Software System Attributes 52
E.1 Reliability 52
E.2 Availability 52
E.3 Security 52
E.4 Maintainability 52
E.5 Portability 53

4 Formal Analysis **54**
A Alloy code 54
A.1 Worlds 61

5 Effort spent **67**

1 Introduction

A Purpose

The **Best Bike Paths (BBP)** system is a comprehensive software platform commissioned by a cyclist association to serve both its members and the broader cycling community.

The fundamental purpose of the system is to facilitate the creation, maintenance, and utilization of a shared repository of bicycle paths. The system addresses two primary operational objectives:

- **Activity Tracking:** BBP enables registered users to monitor personal cycling activities by recording trips and generating detailed statistical reports enriched with environmental variables.
- **Crowdsourced Territorial Mapping:** The platform leverages user participation to update path conditions and identify obstacles. Data acquisition is achieved through manual user reporting or automated detection via mobile device sensors.

A.1 Goals

This section lists the high-level objectives (Goals) that the BBP system must satisfy.

[G1]: Allow registered users to track their cycling activity
The system shall allow users to record their trips, archiving them to maintain a personal history of their cycling activities.

[G2]: Provide detailed statistics on trip performance
For every recorded trip, the system must calculate and display relevant metrics, such as the total distance covered and the average speed maintained by the user.

[G3]: Enrich trip data with contextual information
The system must integrate trip data with meteorological information (weather conditions, temperature, wind speed) retrieved from external services, relative to the time and location of the activity.

[G4]: Enable manual reporting of path status
Registered users must be able to manually insert information regarding path segments, specifying maintenance status (e.g., optimal, requires maintenance) and the presence of obstacles.

[G5]: Automate data collection via mobile sensors
The system must be capable of acquiring data in the background while the user is biking, using GPS for tracking and accelerometer/gyroscope to identify potential road surface anomalies (e.g., potholes).

[G6]: Ensure accuracy of automatically detected data
Since automated detection may generate false positives, the system must require the user to confirm or correct the information detected by sensors before it is validated or published.

[G7]: Enable the publication of collected information
Registered users must be able to explicitly mark their path data (whether manually entered or automatically detected and confirmed) as publishable, making it available to the community. Only publishable information shall be visible to other users and contribute to path scoring.

[G8]: Support route planning for all users
The system must allow any user (even unregistered ones) to input an origin and a destination to visualize available bike paths on a map.

[G9]: Evaluate and rank available paths
When visualizing route options, the system must calculate a score for each path based on its maintenance status and its effectiveness in reaching the destination, sorting results by quality.

[G10]: Detect cycling activity automatically
The system must autonomously identify when the user is cycling (based on speed patterns and device telemetry) to automatically trigger trip recording and sensor data collection.

[G11]: Protect user privacy while enabling collaboration
The system must maintain a clear distinction between private user data (personal trip history and statistics) accessible only to the owner, and publishable path information that contributes to the shared inventory.

Goal	Depends On	Rationale
[G2]	**[G1]**	Trip statistics (distance, speed) cannot be calculated without the primary tracking data established in **[G1]**.
[G3]	**[G1]**	Meteorological data requires a specific time and geographical coordinate anchor provided by the trip record.
[G5]	**[G1]**, **[G10]**	Background sensor collection occurs within the context of a trip (**[G1]**) and is initiated by the auto-detection of activity (**[G10]**).
[G6]	**[G5]**	Validation or correction logic is only applicable once automated sensor data has been captured via **[G5]**.
[G7]	**[G4]**, **[G6]**	Information can only be published once it has been either manually entered (**[G4]**) or automatically sensed and confirmed (**[G6]**).
[G9]	**[G7]**, **[G8]**	Ranking paths requires a pool of published community data (**[G7]**) to overlay onto the basic routing infrastructure (**[G8]**).
[G11]	**[G1]**, **[G7]**	Privacy controls are defined by managing the boundary between private trip history (**[G1]**) and shared community data (**[G7]**).

Table 1: Dependency Mapping of BBP System Goals

B Scope

The scope of the Best Bike Paths (BBP) project encompasses the design and implementation of a mobile-centric architecture supported by a robust cloud backend. The system is responsible for the entire lifecycle of cycling data—including real-time telemetry acquisition, automated anomaly detection via hardware sensors, and the management of a high-integrity community database.

In alignment with the *Problem Frames* approach, we distinguish between:

The World (Environment): The physical and social context in which the system operates, including cyclists, physical bike paths, environmental conditions, and mobile devices.

The Machine (BBP System): The software system itself, including the mobile application, backend services, databases, and integration layers.

B.1 World Phenomena

These are events or states that exist continuously in the real world, independent of the software system.

[WP1]: A cyclist travels along a physical route.

[WP2]: A physical obstacle (e.g., a pothole) exists or appears on a path.

[WP3]: The physical condition of a path deteriorates or improves (status change).

[WP4]: Meteorological conditions (temperature, wind, rain) occur at a specific geographical location.

Goal	[WP1]	[WP2]	[WP3]	[WP4]
[G1]	X			
[G2]	X			
[G3]	X			X
[G4]		X	X	
[G5]	X	X		
[G6]		X		
[G7]		X	X	
[G8]	X			
[G9]		X	X	
[G10]	X			
[G11]	X	X	X	

Table 2: Traceability Matrix: Goals vs World Phenomena

B.2 Shared Phenomena

These describe the interactions between the World (Environment) and the Machine (The BBP System).

Controlled by the World, Observed by the Machine These are inputs or events triggered by actors (Users, Sensors, External Services) that the BBP system detects.

[WM1]: A Registered User initiates or terminates the recording of a trip via the mobile application.

[WM2]: Mobile device sensors transmit telemetry data (GPS coordinates, speed, accelerometer and gyroscope readings) to the system.

[WM3]: A Registered User manually inputs path data (street names, path status).

[WM4]: A Registered User submits a confirmation or correction regarding an automatically detected anomaly (e.g., confirming a pothole).

[WM5]: An Unregistered or Registered User submits a query for a bike path specifying an origin and destination.

[WM6]: The External Weather Service transmits meteorological data upon request.

[WM7]: A Registered User marks path information as publishable.

[WM8]: A Registered User requests to view their trip history.

[WM9]: A User submits authentication credentials (registration or login data).

Controlled by the Machine, Observed by the World These are outputs or actions taken by the BBP system that actors in the world can perceive.

[MW1]: The System visualizes trip statistics (total distance, average speed, performance metrics) to the User.

[MW2]: The System displays weather-enriched trip details to the User.

[MW3]: The System presents a request for verification of a detected anomaly to the Registered User.

[MW4]: The System renders a map visualizing the recommended bike path(s) ranked by score.

[MW5]: The System displays the consolidated status (e.g., "optimal", "requires maintenance") of a specific path segment.

[MW6]: The System notifies the User when automatic trip recording is activated or deactivated based on detected cycling speed.

[MW7]: The System displays a chronological list of the User's previously recorded trips with associated statistics.

[MW8]: The System sends a request for weather conditions to the External Weather Service.

Goal	Controlled by World (Input)								
	[WM1]	[WM2]	[WM3]	[WM4]	[WM5]	[WM6]	[WM7]	[WM8]	[WM9]
[G1]	X	X						X	X
[G2]		X							
[G3]						X			
[G4]			X						X
[G5]		X							X
[G6]				X					X
[G7]			X	X			X		X
[G8]					X				
[G9]					X				
[G10]		X							
[G11]							X		X

Table 3: Traceability Matrix: Goals vs Shared Phenomena (World Inputs)

Goal	Controlled by Machine (Output)							
	[MW1]	[MW2]	[MW3]	[MW4]	[MW5]	[MW6]	[MW7]	[MW8]
[G1]	X						X	
[G2]	X							
[G3]		X						X
[G4]								
[G5]								
[G6]			X					
[G7]								
[G8]				X				
[G9]					X			
[G10]						X		
[G11]					X		X	

Table 4: Traceability Matrix: Goals vs Shared Phenomena (Machine Outputs)

C Definitions, Acronyms, Abbreviations

C.1 Definitions

- **Best Bike Paths (BBP):** The software system to be developed, commissioned by a cyclist association to manage a collaborative inventory of bike paths.
- **Registered User:** A user who has created an account on the platform. They can record trips, report anomalies, and access their personal history.
- **Unregistered User:** A user who accesses the platform without an account. They are limited to viewing the map and planning routes without saving data.
- **Trip:** A recorded session of cycling activity, containing geographical data, time duration, and performance statistics.
- **Automated Mode:** A system feature that automatically detects cycling activity and records trip data, including sensor-based anomaly detection, without manual intervention.
- **Manual Mode:** A method of data entry where the user manually inputs details about a path segment, such as its maintenance status or specific obstacles.
- **Confirmation:** The action taken by a Registered User to validate a potential anomaly detected by the system sensors, marking it as a verified obstacle or discarding it as a false positive.
- **Anomaly:** An irregularity detected on the road surface (e.g., a pothole or rough terrain) by the mobile device's sensors (accelerometer and gyroscope).

- **Path Score:** A numerical value calculated by the system to rank bike paths based on their maintenance status and efficiency in reaching a destination.
- **Publishable Information:** Data (trips, anomalies, or path status) that a Registered User has explicitly marked as public, making it visible to the entire community.

C.2 Acronyms

- **BBP:** Best Bike Paths
- **GPS:** Global Positioning System
- **API:** Application Programming Interface
- **GDPR:** General Data Protection Regulation
- **UI:** User Interface
- **UML:** Unified Modeling Language
- **BPMN:** Business Process Model and Notation

C.3 Abbreviations

- **G**n**:** Goal number n
- **WP**n**:** World Phenomenon number n
- **WM**n**:** Shared Phenomenon Controlled by the World, Observed by the Machine number n
- **MW**n**:** Shared Phenomenon Controlled by the Machine, Observed by the World number n
- **S**n**:** Scenario number n
- **UC**n**:** Use Case number n
- **R**n**:** Functional Requirement number n
- **PR**n**:** Performance Requirement number n
- **D_ASS**n**:** Domain Assumption number n
- **CON**n**:** Constraint number n
- **DC**n**:** Design Constraint number n
- **SSA**n**:** Software System Attribute number n

D Revision History

- December 23, 2025: version 1.0, first release

E Reference Documents

- Specification document: "Assignment RDD AY 2025-2026"
- Official alloy documentation: https://alloytools.org/documentation.html
- UML official specification: https://www.planttext.com/
- BPMN official specification: https://bpmn.io/

F Document Structure

The document is organized into five main chapters, structured as follows:

- **Section 1: Introduction**
  This chapter provides a high-level overview of the Best Bike Paths (BBP) project, defining its scope and primary objectives. It establishes the terminology used throughout the document by listing definitions and acronyms. Additionally, it tracks the document's evolution through the revision history and outlines the organization of the subsequent sections.
- **Section 2: Overall Description**
  This section defines the context in which the system operates. It illustrates the interaction between users and the system through realistic scenarios and identifies the various actors involved. Furthermore, it provides a general overview of the product's functions, while detailing the hardware and software constraints, dependencies, and assumptions that influence the design.
- **Section 3: Specific Requirements**
  This chapter represents the core of the specification, detailing the functional and non-functional requirements of the system. The functional aspects are modeled and visualized using UML diagrams (Use Case and Sequence diagrams) to define system behavior. Finally, a traceability matrix maps these requirements back to the initial goals to ensure complete coverage.
- **Section 4: Formal Analysis using Alloy**
  This section presents a formal verification of specific critical aspects of the system. It describes the Alloy model constructed to validate the consistency and correctness of the requirements, discussing the results of the assertions checks and the generated worlds.
- **Section 5: Effort Spent**
  The final section provides a breakdown of the time and effort invested by each group member in the analysis and drafting of this document.

2 Overall Description

A Product perspective

A.1 Scenarios

1. **User registration and first access**
   Alice is a cycling enthusiast who wants to keep track of her daily commutes and contribute to the community. She downloads the BBP application and selects the option to sign up. She inputs her personal details (username, email, and password). After validating her email via a link sent to her inbox, she logs in for the first time. The system welcomes her and shows her personal dashboard, which is currently empty of trips.

2. **Recording a personal trip with statistics**
   Marco, a registered user, decides to go for a ride on Sunday morning. He opens the BBP app and presses the "Start Recording" button. He puts the phone in his pocket and enjoys his ride. Once he reaches his destination, he takes out his phone and presses "Stop". The system immediately processes the data and presents a summary of the trip, showing the total distance (45 km) and his average speed (22 km/h). Additionally, since the internet connection is available, the system automatically enriches this record with meteorological data from the area during that timeframe (Sunny, 25°C, Wind 10km/h). Marco saves this trip to his personal archive.

3. **Automated data collection and user confirmation (Positive Case)**
   Marco, a registered user, is riding his bike to work with the "Automated Mode" enabled in the settings. The system detects that his movement speed is compatible with a bicycle and automatically starts monitoring the sensors. During the ride, the smartphone's accelerometer detects a significant vertical jolt compatible with a pothole. When Marco stops biking, the system sends him a notification: "We detected a potential obstacle during your ride. Please verify." Marco opens the notification and sees a map with a pin on the location where the jolt occurred. He confirms that there was indeed a pothole. The system marks the information as valid and publishable.

4. **Handling a False Positive in Automated Mode**
   Marco is using the BBP app in automated mode. While riding, he decides to jump off a curb to enter a park. The accelerometer detects a strong vibration and flags it as a potential anomaly. At the end of the ride, the system asks Marco to confirm a "Severe Obstacle" at that specific location. Marco recognizes the spot on the map and knows it wasn't a road defect, but his own action. He presses "Reject". The system discards the data, ensuring that no false information is added to the database.

5. **Manual insertion of path status**
   Alice is walking her bike along a path she usually takes, but she notices

that a section of the track is currently flooded and muddy due to recent rain. She decides to warn other cyclists immediately. She opens the BBP app, logs in, and selects the "Report Path" function. She manually selects the street segments on the map, sets the status to "Requires Maintenance" and adds a note about the mud. She sets the info as "Publishable" and submits it.

6. **Browsing Trip History and Statistics**
   Marco wants to check his progress for the month. He opens the BBP app and navigates to the "My History" section. The system retrieves his list of past trips. He selects a trip from two weeks ago. The system displays the detailed map of the route he took, the weather conditions of that day, and his speed graph. Marco notices he is getting faster and closes the app with satisfaction.

7. **Route planning and visualization (Unregistered User)**
   Marco is a tourist visiting the city and wants to go from the Central Station to the City Park by bike. He is not registered on BBP, but he can still access the "Explore" section. He enters his origin (Central Station) and destination (City Park). The system calculates possible routes and visualizes them on the map. It suggests two paths: Path A and Path B. The system highlights Path A as the best option because it has a higher score. Marco sees that Path B is shorter but marked as "Poor Condition", so he chooses Path A.

8. **Automatic Activity Detection (Context Recognition)**
   Alice leaves her office and jumps on her bike to go home, forgetting to open the BBP app. She starts pedaling vigorously. After a few minutes, the system detects a sustained speed of 18 km/h and a movement pattern typical of cycling. The app sends a silent notification: "Cycling activity detected. Recording started". Alice arrives home and finds that her trip has been correctly tracked from the moment the system recognized the activity.

9. **Publishing Private Data**
   Marco has recorded several trips in "Private" mode because he didn't want to share his location initially. However, he realizes that one of his routes covers a newly paved bike lane that isn't on the map yet. He goes into his "Trip History," selects that specific route, and clicks "Make Path Info Publishable". The system extracts the path quality data and updates the public inventory, allowing other users to benefit from this new information.

10. **Viewing Path Details and Scores**
    Alice is planning a weekend ride and is looking at the map of her neighborhood. She sees a bike path marked in green. She taps on it to see the details. The system displays the "Path Score" (4.8/5) and the status "Optimal". She sees another path marked in yellow with a score of (3.0/5). By clicking on it, she reads that the status is "Medium" because

of frequent tree roots. Based on this detailed information, she plans her route to avoid the yellow path.

ID	**Scenario Title**	**Goals Covered**	**Shared Phenomena Covered**
S1	User registration and first access	[G1]	[WM9]
S2	Recording a personal trip with statistics	[G1], [G2], [G3]	[MW1], [MW8], [WM6], [MW2]
S3	Automated data collection and user confirmation (Positive Case)	[G5], [G6]	[WM2], [MW3], [WM4]
S4	Handling a False Positive in Automated Mode	[G6]	[WM2], [MW3], [WM4]
S5	Manual insertion of path status	[G4], [G7]	[WM3], [WM7]
S6	Browsing Trip History and Statistics	[G1], [G2], [G3]	[WM8], [MW7], [MW1], [MW2]
S7	Route planning and visualization (Unregistered User)	[G8], [G9]	[WM5], [MW4]
S8	Automatic Activity Detection (Context Recognition)	[G10], [G1]	[WM2], [MW6]
S9	Publishing Private Data	[G7], [G11]	[WM8], [WM7], [MW5]
S10	Viewing Path Details and Scores	[G9]	[MW4], [MW5]

Table 5: Scenarios Traceability Matrix

A.2 Class Diagram

The domain model distinguishes between RegisteredUsers, who track activities and report data, and UnregisteredUsers restricted to navigation. The central Trip entity records cycling history by aggregating GPSCoordinates and calculating TripStatistics, while linking to SensorAnomaly for automated defect detection and Weather Data for environmental context.

Path management relies on BikePathSegments, which maintain status and Obstacle history derived from user-submitted PathReports. These inputs determine the PathScore, a metric used alongside the external MapService to process user RouteRequests and generate optimized RouteResults, ensuring recommendations reflect the most current community-validated road conditions.

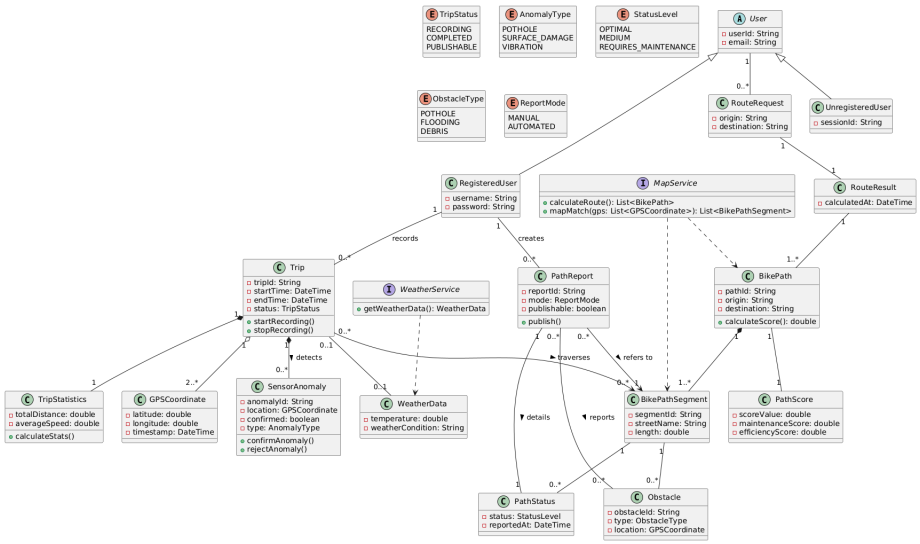


Figure 1: Class Diagram

A.3 State Diagrams

For the BBP system, two key entities exhibit complex state-dependent behavior that requires formal modeling: the **Trip** (representing a cycling session) and the **SensorAnomaly** (representing a potential road defect detected automatically).

1. Trip Lifecycle The `Trip` entity is the core data unit of the application. Its lifecycle manages the transition from active recording to data processing, storage, and eventual publication.

Description of States:

- **Idle:** The initial state where the system is ready but not recording.
- **Recording:** The user is actively cycling. This state can be entered manually or automatically via activity detection. In this state, the system continuously tracks GPS and sensor data.
- **Processing:** Once the recording stops (manually or via timeout), the system calculates statistics (speed, distance) and attempts to fetch weather data.
- **Private (Stored):** The trip is saved in the user's history but is visible only to them.
- **Public (Published):** The user has explicitly chosen to share the trip data, updating the community path inventory.

Diagram Logic: The transition from *Idle* to *Recording* is triggered by `startRecording()` (Manual) or when `speed > threshold` (Automated). The transition to *Processing* occurs upon `stopRecording()` or when movement ceases for a defined time (timeout). Finally, the `makePublishable()` event transitions the trip from *Private* to *Public*, triggering the anonymization process.

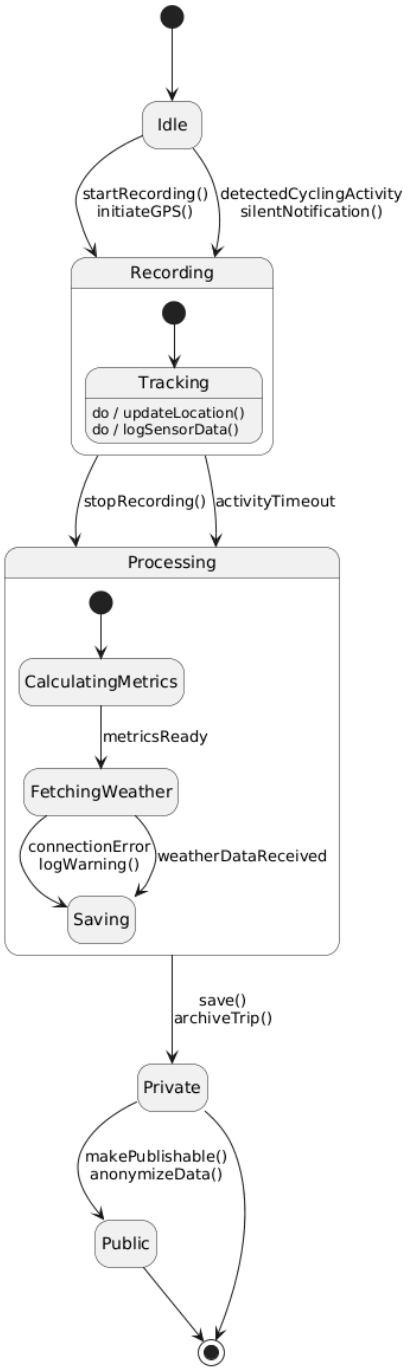


Figure 2: State Machine Diagram for the Trip entity

2. Sensor Anomaly Lifecycle The `SensorAnomaly` entity represents the crowdsourcing validation workflow. It handles the lifecycle of data detected by hardware sensors to ensure false positives are filtered out before polluting the public database.

Description of States:

- **Pending (Verification):** The initial state of an anomaly immediately after it has been detected by the system sensors and logged locally. It awaits user interaction.
- **Confirmed:** The user has validated the anomaly as a real obstacle (e.g., pothole). This state corresponds to valid knowledge added to the system.
- **Rejected:** The user has marked the anomaly as a false positive (e.g., jumping a curb). The data is discarded and removed.

Diagram Logic: The lifecycle begins when a `verticalShockDetected` event occurs, which triggers the instantiation of the `SensorAnomaly` entity directly in the `Pending` state. The entity remains in `Pending` until the user interacts via the "Verify Anomalies" interface. The `confirm()` event transitions the state to `Confirmed`, creating a corresponding `PathReport`, while `reject()` transitions it to `Rejected`, causing the deletion of the temporary data.

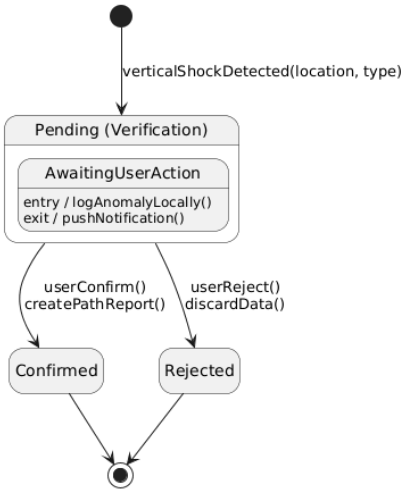


Figure 3: State Machine Diagram for the SensorAnomaly entity

B Product functions

B.1 User Registration (Sign Up)

This functionality allows Unregistered Users to create a personal account to access the system's full features. The user provides personal details such as username, email, and password. The system verifies the validity of the input data (e.g. ensuring the email is not already registered). Upon successful validation, the system creates a new account and stores the user's credentials.

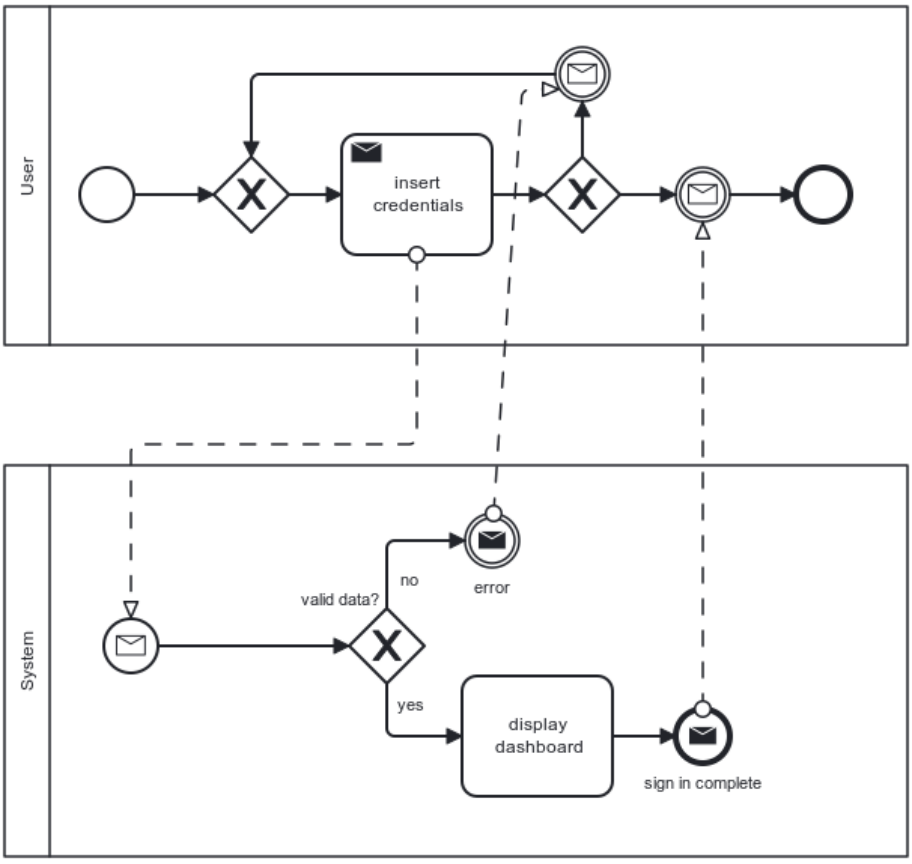


Figure 4: User Registration BPMN

B.2 User Login

This functionality grants Registered Users access to their personal area. The user inputs their credentials, and the system authenticates them against the database. If the authentication is successful, the user is redirected to their personal dashboard; otherwise, an error message is displayed.

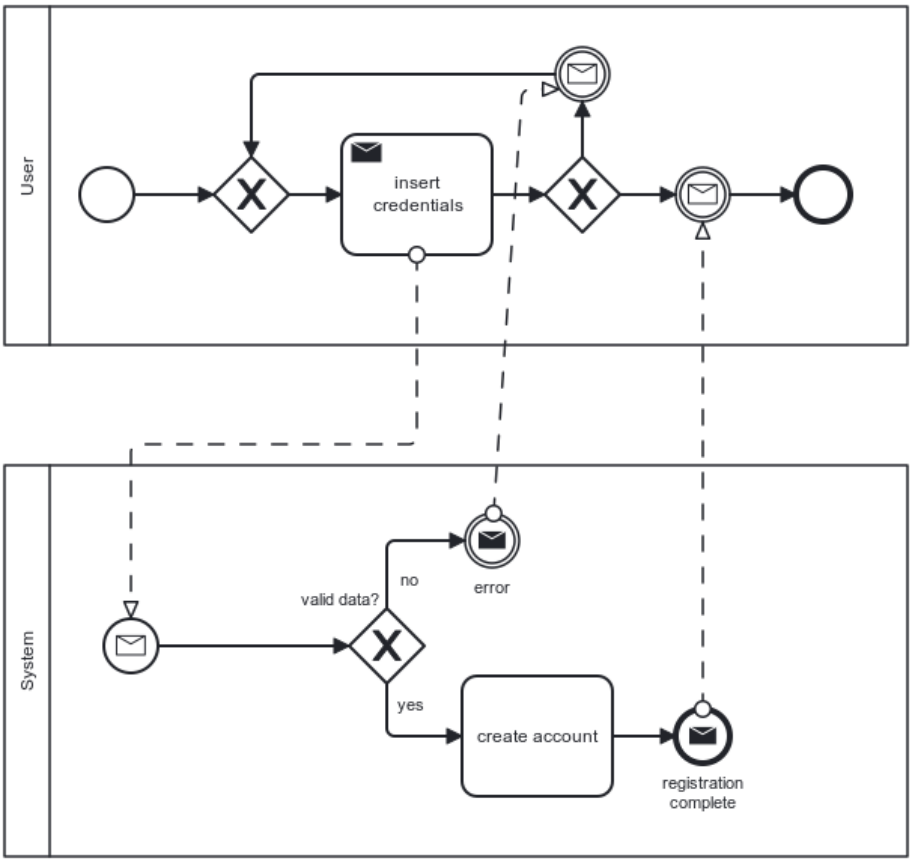


Figure 5: User Login BPMN

B.3 Record Personal Trip

This functionality allows Registered Users to manually track their cycling activity. The user initiates the recording via the application interface. During the ride, the system continuously tracks the user's geolocation (GPS) and time. The recording continues until the user explicitly stops it, at which point the raw data is saved locally for processing.

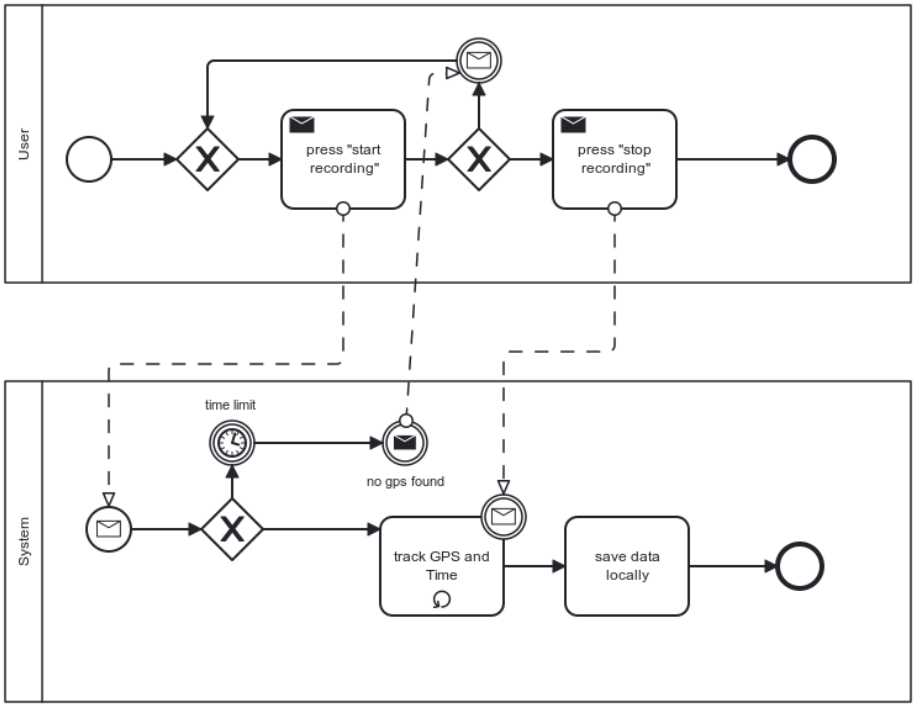


Figure 6: Record Personal Trip BPMN

B.4 Trip Statistics & Weather Enrichment

Once a trip is completed, this functionality processes the raw data to calculate performance metrics such as total distance and average speed. Furthermore, the system connects to an External Weather Service to retrieve meteorological conditions, specific to the time and location of the trip. These details are then displayed to the user and archived in their history.

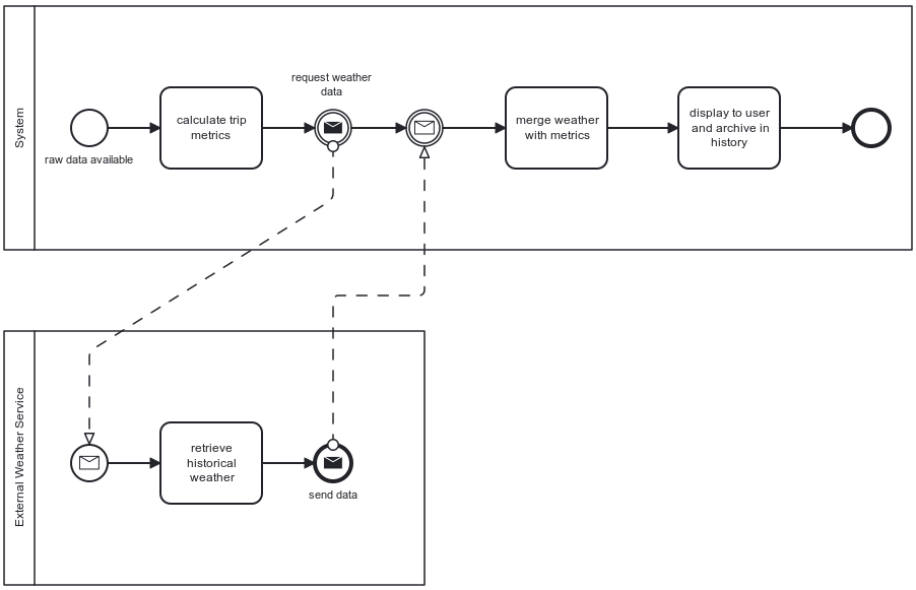


Figure 7: Trip Statistics & Weather Enrichment BPMN

B.5 Automated Data Collection

This background functionality automatically detects when the user is cycling based on movement speed. Once activated, it collects data from the mobile device's sensors to identify road anomalies, such as potholes. Potential obstacles are flagged internally by the system for later verification.

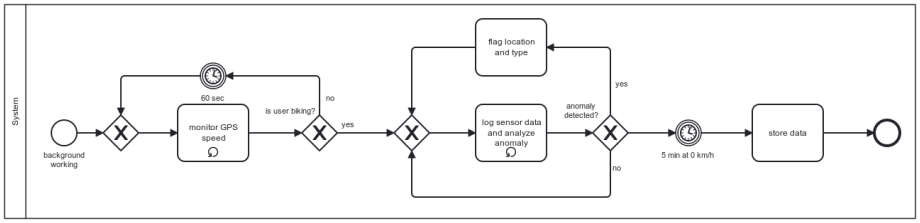


Figure 8: Automated Data Collection BPMN

B.6 Verify & Confirm Detected Data

This functionality ensures the accuracy of automatically collected data. The system presents the user with a list or map of potential anomalies detected during the ride. The user must explicitly confirm if the anomaly was real or reject it (a false positive). Only confirmed data is stored as valid path information.

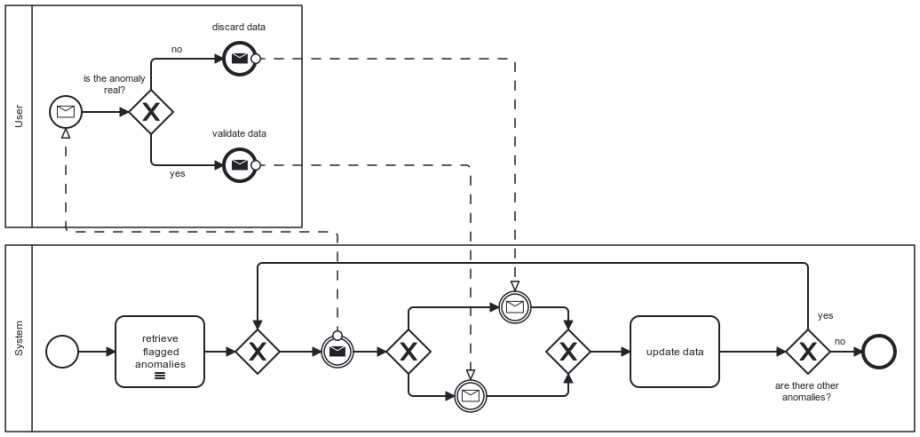


Figure 9: Verify & Confirm Detected Data BPMN

B.7 Manual Path Reporting

Registered Users can manually insert information about a specific path segment. The user selects a location on the map, specifies the status and indicates any obstacles. This data is saved to the user's personal records and can be optionally published.

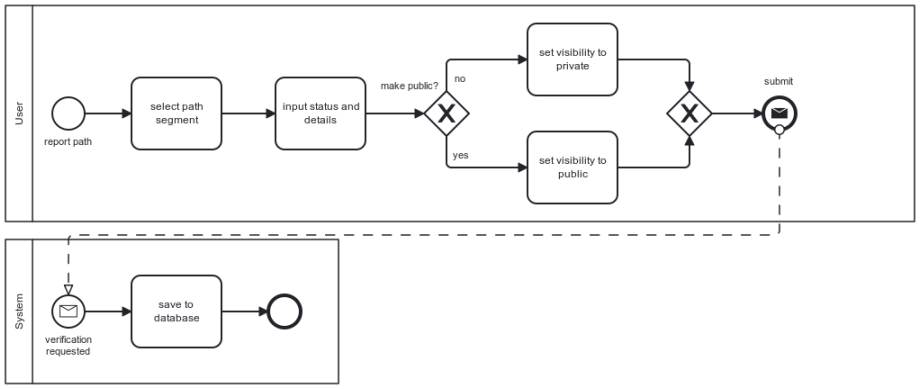


Figure 10: Manual Path Reporting BPMN

B.8 Publish Path Information

This functionality allows Registered Users to contribute to the shared inventory. The user selects validated data (either manually reported or automatically confirmed) from their personal history and marks it as "Publishable". The system updates the public database, making this information available for other users' route planning.

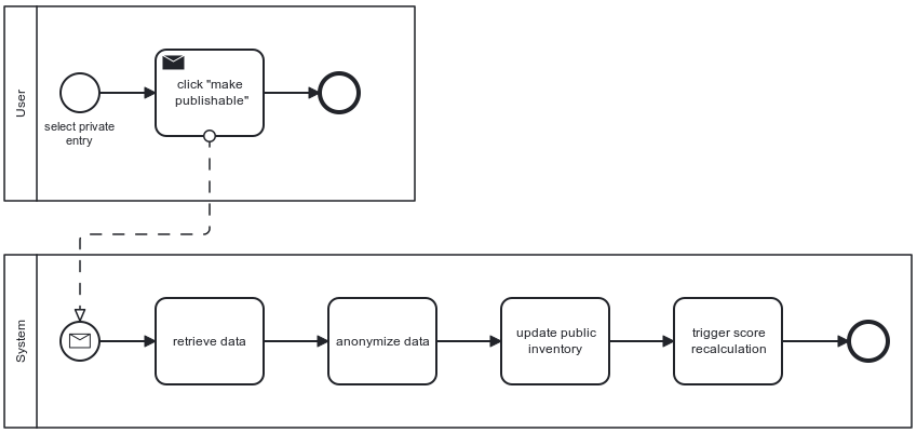


Figure 11: Publish Path Information BPMN

B.9 Route Planning & Scoring

This functionality allows any user (Registered or Unregistered) to find the best bike path between two points. The user inputs an origin and a destination. The system calculates available routes and assigns a score to each based on path status and efficiency. The results are visualized on a map, highlighting the highest-scored path.

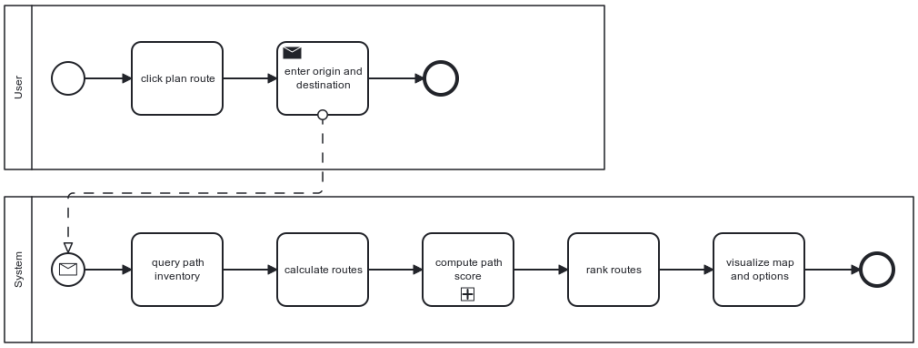


Figure 12: Route Planning & Scoring BPMN

B.10 View Statistics & History

This functionality enables Registered Users to access their personal archive of cycling activities. The user accesses the "History" section to view a chronological list of past trips. For each entry, the system provides a summary (date, distance, duration). By selecting a specific trip, the user can access detailed information, including the route map, performance charts (speed/elevation), and the associated weather conditions recorded at the time of the trip.

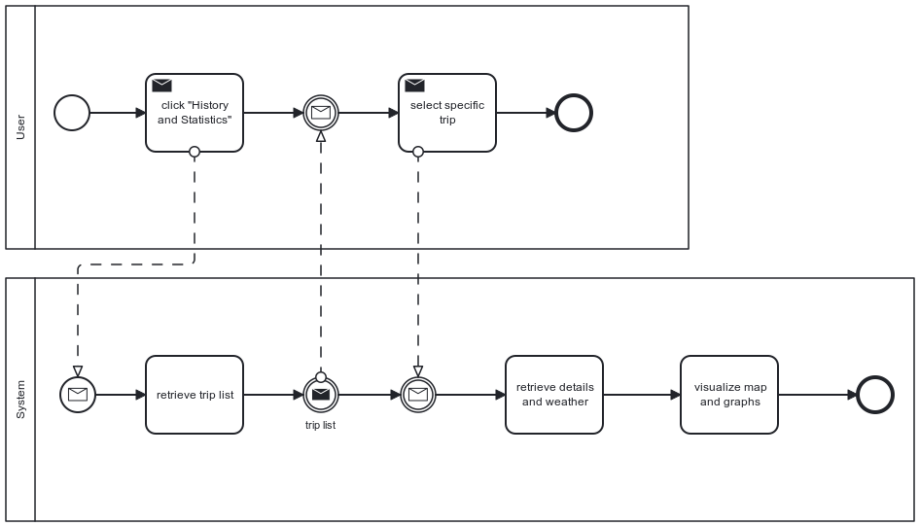


Figure 13: View Statistics & History BPMN

C User characteristics

This section describes the intended audience of the Best Bike Paths (BBP) system. The user population consists of individuals with varying degrees of technical proficiency and cycling habits.

The users are categorized into two main groups:

1. **Registered Users (Contributors & Trackers)**
    - *Profile:* These users are likely regular cyclists, ranging from daily commuters to sports enthusiasts. They are motivated by the desire to track their performance (quantified self) and to contribute to the community by improving map quality.
    - *Technical Proficiency:* They are assumed to possess average to high familiarity with smartphone technologies, GPS navigation apps, and fitness tracking tools.
    - *Physical Context:* A critical characteristic of this group is that they interact with the system while performing a physical activity. Therefore, interactions with the application must be minimal and quick.
    - *Key Needs:* Accuracy of statistics, battery efficiency, and minimal distraction during the ride.
2. **Unregistered Users (Explorers)**
    - *Profile:* This group includes casual cyclists, tourists, or citizens who need occasional guidance. They are not interested in tracking personal metrics but solely in reaching a destination safely.
    - *Technical Proficiency:* Varies widely from low to high.
    - *Key Needs:* Immediate access to routing information without the friction of a sign-up process. They prioritize the safety and quality of the suggested path over performance metrics.

D Assumptions, dependencies and constraints

D.1 Domain Assumptions

Domain assumptions are facts about the environment that are considered true for the system to function correctly. If these assumptions do not hold, the system may not be able to satisfy its requirements.

[D_ASS1] GPS Availability: It is assumed that the Global Positioning System (GPS) signal is available and sufficiently accurate in the areas where the system is used to allow for meaningful path reconstruction.

[D_ASS2] Device Sensor Capability: It is assumed that the users' smartphones are equipped with a functioning accelerometer and gyroscope capable of detecting vibrations and orientation changes.

[D_ASS3] Device Placement: For the "Automated Mode" to function correctly, it is assumed that the user carries the smartphone in a position that allows the sensors to perceive road irregularities (e.g., firmly attached to the handlebar or in a pocket close to the body), rather than in a suspended bag where vibrations would be dampened.

[D_ASS4] Distinguishability of Speed: It is assumed that cycling speeds are sufficiently distinct from walking (too slow) and driving (too fast) to allow the system to infer cycling activity with a reasonable degree of confidence, triggering the automatic recording.

[D_ASS5] User Integrity: It is assumed that the majority of users provide truthful manual reports and confirmations regarding path conditions.

D.2 Dependencies

The BBP system relies on external systems and services to provide full functionality.

[DEP1] External Map Service: BBP does not create cartography from scratch. It depends on a third-party Map Provider (e.g., OpenStreetMap, Google Maps Platform) to render maps and provide the underlying road network topology for visualization.

[DEP2] External Weather Service: The system depends on the availability and reliability of an external API to fetch real-time and historical meteorological data based on geolocation and timestamps.

[DEP3] Mobile Data Connectivity: The system depends on the mobile network ($4G/5G$) to download map tiles, fetch weather data, and synchronize trip data with the server. However, core tracking features must be resilient to temporary loss of connectivity.

D.3 Constraints

Constraints are limitations imposed on the design and implementation of the solution.

[CON1] Regulatory & Privacy: As the system collects sensitive geolocation data and personal history, it must strictly adhere to data protection regulations (e.g., GDPR). Private trips must remain inaccessible to other users unless explicitly published.

[CON2] Hardware Heterogeneity: The system must function across a wide range of Android and iOS devices with varying sensor sensitivity and GPS hardware quality.

[CON3] Connectivity Limitations: The application must support offline buffering. If a user cycles in a remote area without signal, the trip data and sensor logs must be stored locally and synchronized once connectivity is restored.

3 Specific Requirements

This section contains the detailed requirements for the Best Bike Paths (BBP) system. It provides a comprehensive description of the external interfaces, functional requirements, and non-functional constraints necessary for the development team to implement the solution.

A External Interface Requirements

This subsection describes how the system interfaces with users, hardware, other software, and communications.

A.1 User Interfaces

The system is primarily accessed via a mobile application (Android/iOS). The user interface must be designed to be minimal and high-contrast to ensure legibility while cycling (Safety Critical).

- **Map Visualization:** The UI shall display a map as the central element, rendering paths with color codes indicating their status (e.g., Green for "Optimal", Red for "Poor").
- **Controls:** Buttons for starting/stopping trips and reporting anomalies must be large and easily tappable to facilitate interaction during physical activity.
- **Notifications:** The system shall use non-intrusive notifications (audio cues or heads-up alerts) for automated detection alerts to minimize user distraction.

A.2 Hardware Interfaces

The mobile application interacts directly with the smartphone's hardware sensors to fulfill the automated data collection goals.

- **GPS Receiver:** Used to track geolocation (latitude, longitude, altitude) and calculate real-time speed.
- **Accelerometer & Gyroscope:** Used to detect vertical accelerations and device orientation changes indicative of road anomalies (e.g., potholes) or specific cycling movement patterns.
- **Local Storage:** The device's file system is used to buffer trip data and sensor logs temporarily when an internet connection is unavailable.

A.3 Software Interfaces

The BBP system interacts with the following external software entities:

- **Map Service API** (e.g., Google Maps, OpenStreetMap): Used to retrieve map tiles and access the underlying road network topology.
- **Weather Service API:** Used to fetch meteorological data (Temperature, Wind Speed, Weather Condition) based on the specific timestamp and location coordinates of a trip.
- **Operating System** (Android/iOS): The app interfaces with the host OS to manage permissions (Location Always On, Background Processing) and utilize system notification services.

A.4 Communication Interfaces

- **Protocol:** Communication between the Mobile Client and the Backend Server shall occur over HTTPS to ensure data confidentiality and integrity.
- **Data Format:** Data exchange (e.g., trip uploads, route requests) shall use JSON format over RESTful APIs.

B Functional Requirements

B.1 Use case diagrams

1. **Unregistered User**

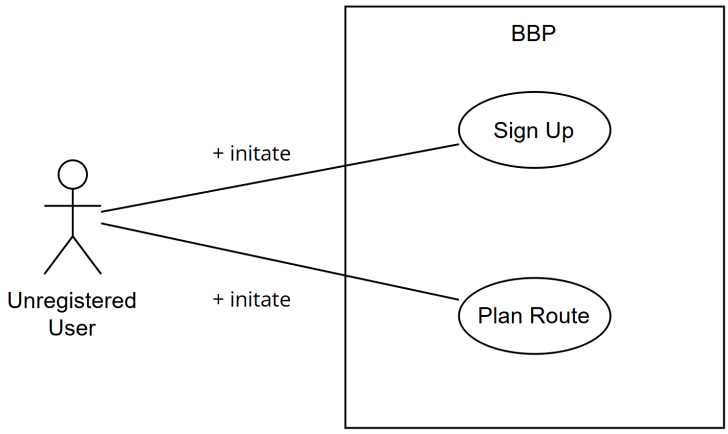


Figure 14: Unregistered User - Use Case Diagram

2. **Registered User**

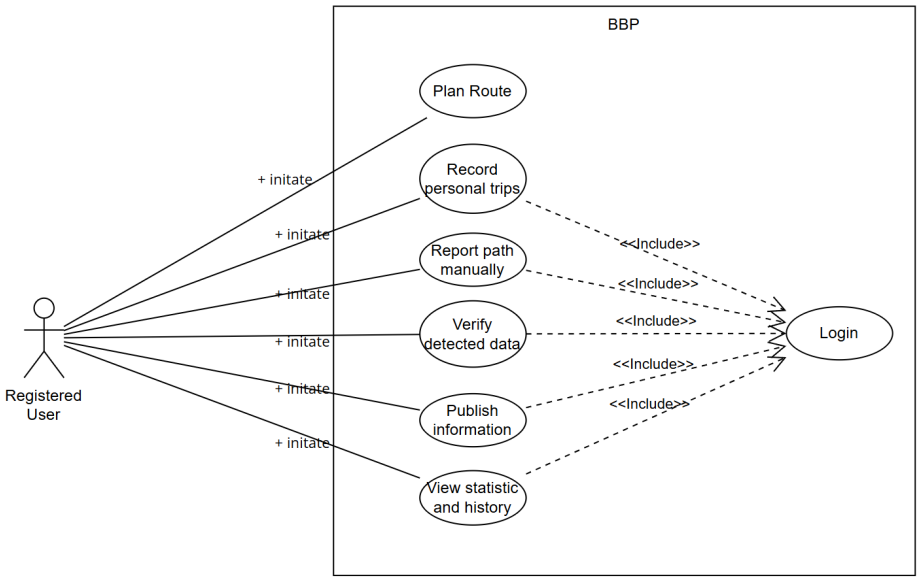


Figure 15: Registered User - Use Case Diagram

B.2 Use Case Tables and Sequence Diagrams

ID	UC1
Name	Sign Up
Actor	Unregistered User
Entry conditions	The User has downloaded the application, opened it, and selected the "Sign Up" option.
Input	`username`, `email`, `password`
Events flow	1. The application displays the Sign Up form. 2. The Unregistered User inserts the required data: username, email, and password. 3. The Unregistered User clicks the "Register" button. 4. The System checks if the email is already in use and validates the password strength. 5. The application displays a confirmation message informing the User that a verification email has been sent. 6. The Unregistered User opens their email client and clicks on the verification link received from BBP. 7. The application displays a success message confirming the account activation. 8. The application redirects the User to the app Home screen.
Exit conditions	• The User account is created and marked as active in the database. • The User is redirected to the Home page.
Output	• Registration confirmation email sent to the User. • New Account record stored in the database.
Exceptions	**Email already exists:** If the email is already registered, the application displays an error message on the Sign Up form inviting the User to log in or use a different email.

Table 6: Use Case 1: Sign Up

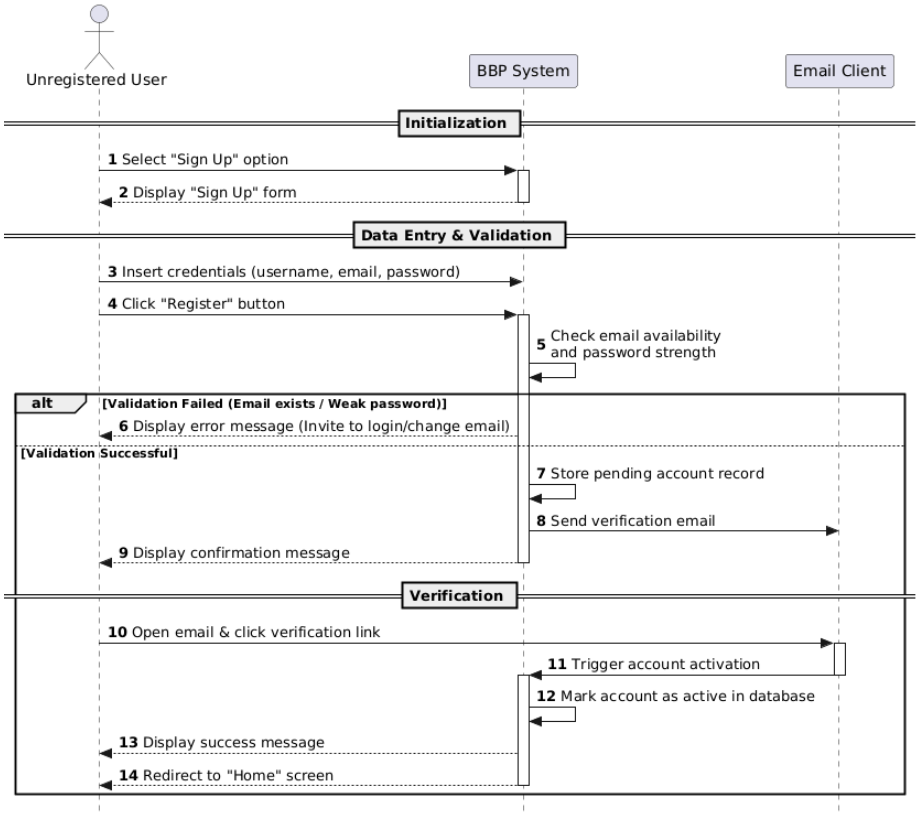


Figure 16: Sign Up

ID	UC2
Name	Log In
Actor	Registered User
Entry conditions	The User has opened the application but is not currently authenticated. The User already possesses a registered account.
Input	`email` (or `username`), `password`
Events flow	1. The application displays the initial screen with the "Log In" option. 2. The User selects the "Log In" option. 3. The application displays the credential entry form. 4. The User inserts their email (or username) and password. 5. The User clicks the "Log In" button. 6. The System verifies the correctness of the credentials against the database. 7. The System authenticates the User and generates a session. 8. The application redirects the User to the main Dashboard (Home Page).
Exit conditions	• The User is successfully authenticated. • The User is redirected to the main Dashboard.
Output	• Access granted to personalized features. • User session active.
Exceptions	**Invalid Credentials:** If the User enters an incorrect email or password, the application displays an error message (e.g., "Invalid credentials") and invites the User to retry, keeping them on the Log In screen.

Table 7: Use Case 2: Log In

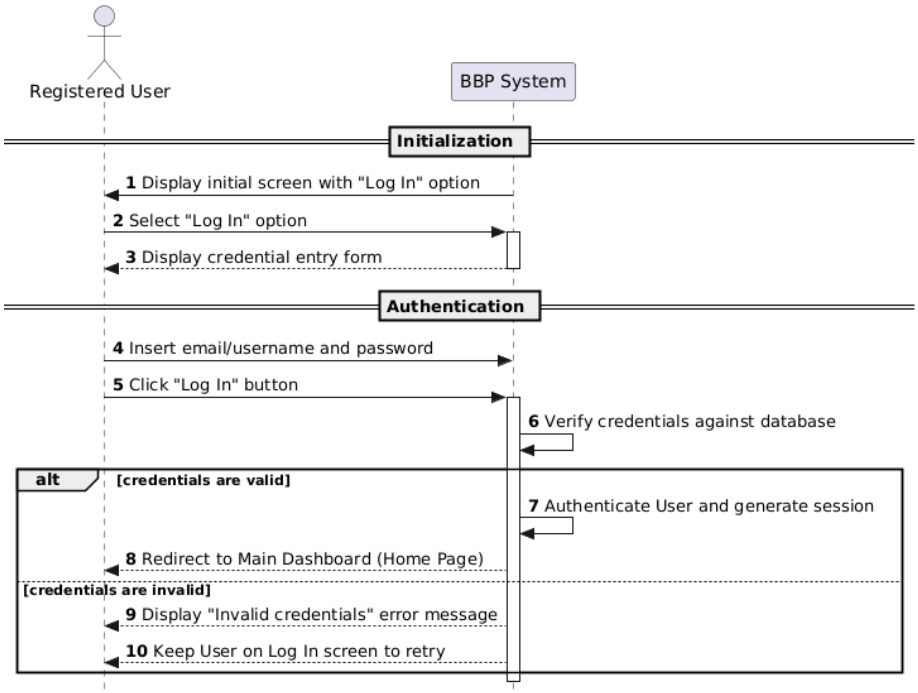


Figure 17: Log In

ID	UC3
Name	Plan Route
Actor	Unregistered User, Registered User
Entry conditions	The User has opened the application and accessed the "Explore" section.
Input	`origin`, `destination`
Events flow	1. The application displays a map view with a search bar. 2. The User enters the address for the "Origin" and "Destination". 3. The User clicks the "Find Route" button. 4. The System calculates available routes based on the Path Inventory and external map data. 5. The application displays the map with the calculated routes drawn on it. The route with the highest score is highlighted in green. 6. The application shows a summary card with details (distance, estimated time, and Path Score) for the best route. 7. The User selects a specific route to view detailed segment information.
Exit conditions	• The optimal path is visualized on the user's screen. • The User can proceed to navigation.
Output	• Map visualization of the routes. • Route statistics (Path Score, Status, Distance).
Exceptions	**No Path Found:** If no route connects the points, the application displays a generic error message ("No bike paths found") and keeps the User on the search view.

Table 8: Use Case 3: Plan Route

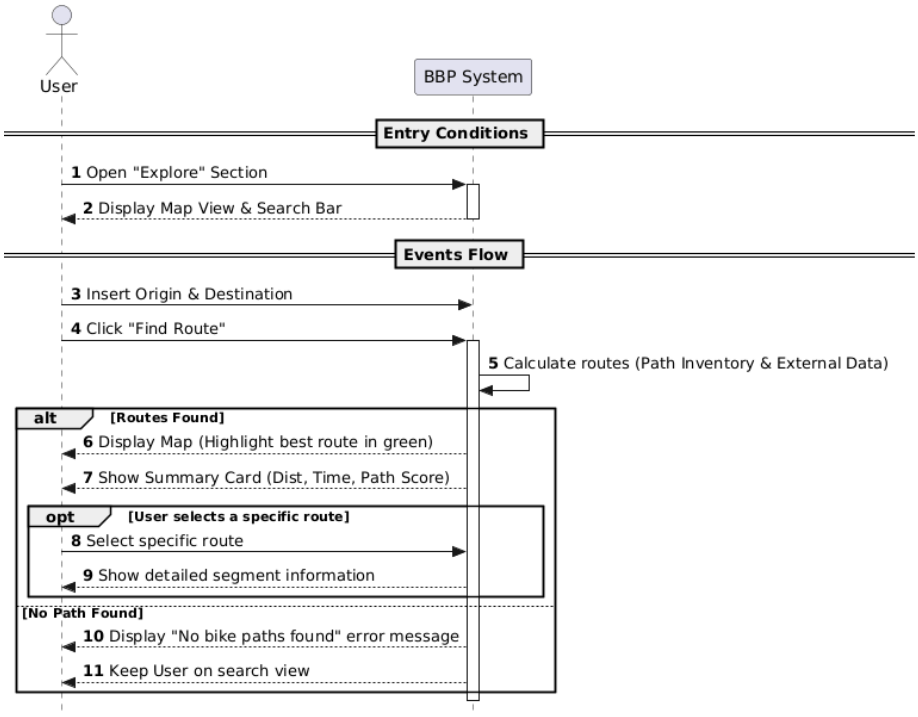


Figure 18: Plan Route

ID	UC4
Name	Record Personal Trip
Actor	Registered User
Entry conditions	The User has logged in and is ready to start a cycling activity.
Input	User interaction (Start/Stop), GPS Data
Events flow	1. The User taps the "Start Recording" button on the dashboard. 2. The System begins tracking GPS coordinates and time in the background. 3. The User cycles to their destination. 4. The User taps the "Stop Recording" button. 5. The System processes the raw GPS data to calculate Trip Statistics (Total Distance, Average Speed). 6. The System requests current meteorological data (Temperature, Weather Condition) from the External Weather Service based on the trip's location and time. 7. The System merges the statistics with the weather data. 8. The application displays a Trip Summary screen with the map, stats, and weather info. 9. The User taps "Save" to archive the trip.
Exit conditions	• The trip is saved in the User's personal history (Private status). • Statistics are updated.
Output	• Trip Summary visualization. • New Trip entity stored in the database.
Exceptions	**No GPS/Connectivity:** If GPS is weak, the System notifies the user. If internet is missing at the end of the trip, the System saves the raw data locally and retries fetching weather data once connectivity is restored.

Table 9: Use Case 4: Record Personal Trip

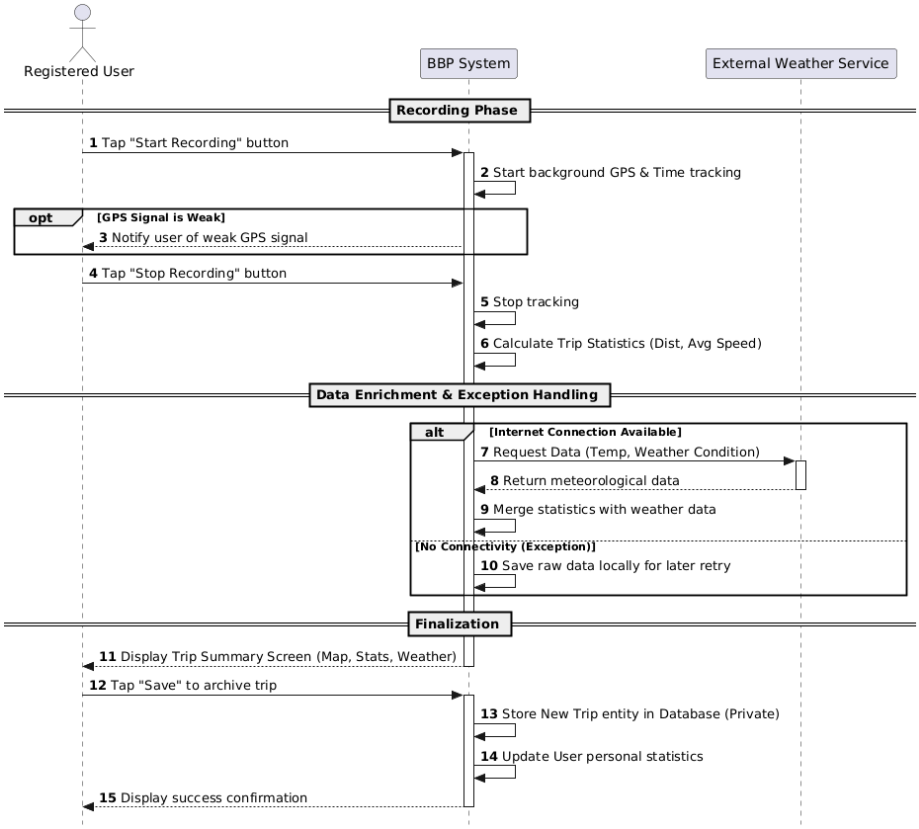


Figure 19: Record A Personal Trip

ID	UC5
Name	Automated Data Collection
Actor	Registered User (Passive), System (Active)
Entry conditions	The User has logged in, enabled "Automated Mode" in settings, and starts cycling.
Input	GPS Speed, Accelerometer, Gyroscope
Events flow	1. The System detects that the device speed exceeds the cycling threshold for a sustained period. 2. The application triggers a silent notification: "Cycling activity detected. Recording started." 3. The System monitors sensor data (accelerometer/gyroscope) in the background. 4. The System detects a significant vertical shock (potential anomaly). 5. The System logs the anomaly type (e.g., Vibration/Pothole), timestamp, and GPS location internally. 6. The User stops moving (speed drops to 0) for more than 5 minutes. 7. The System stops the recording session. 8. The application sends a push notification: "We detected potential obstacles during your ride. Tap to verify."
Exit conditions	• Trip data is stored locally. • Potential anomalies are flagged for verification (Pending status).
Output	• Silent Notification (Start). • Push Notification (End).
Exceptions	**Low Battery:** If the device battery drops below a certain threshold, the System stops background monitoring to preserve power and saves the current state.

Table 10: Use Case 5: Automated Data Collection

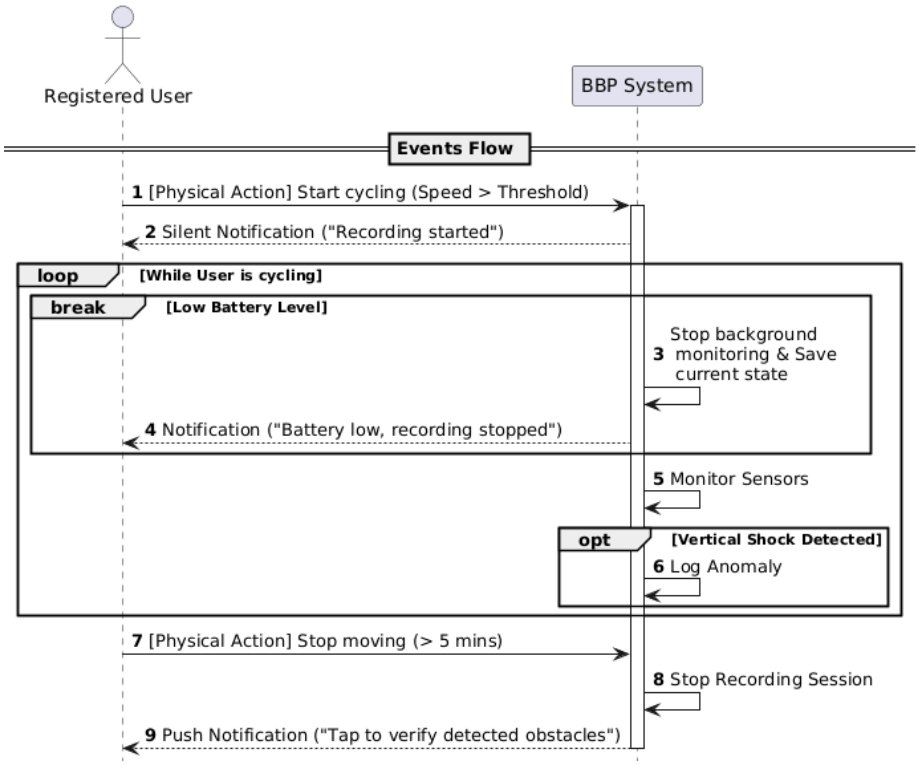


Figure 20: Automated Data Collection

ID	UC6
Name	Verify Detected Data
Actor	Registered User
Entry conditions	The User taps on the "Verify Anomalies" notification from UC5 or opens a trip with pending anomalies.
Input	User Confirmation (Confirm/Reject)
Events flow	1. The application displays the map of the recorded trip with pins marking detected anomalies. 2. The User taps on a specific anomaly pin. 3. The application displays a popup: "Potential Pothole detected here. Confirm?" 4. The User evaluates the location and clicks "Confirm". 5. The application changes the anomaly status to "Confirmed". 6. The System asks if there are other anomalies to review. 7. The User reviews all items and finally clicks "Save Verification".
Exit conditions	• Confirmed anomalies are stored as valid Path Reports. • Rejected anomalies are discarded.
Output	Updated local trip data with validated incidents.
Exceptions	**False Positive:** If the User clicks "Reject" (e.g., they jumped a curb), the application removes the pin from the map and deletes the temporary anomaly data.

Table 11: Use Case 6: Verify Detected Data

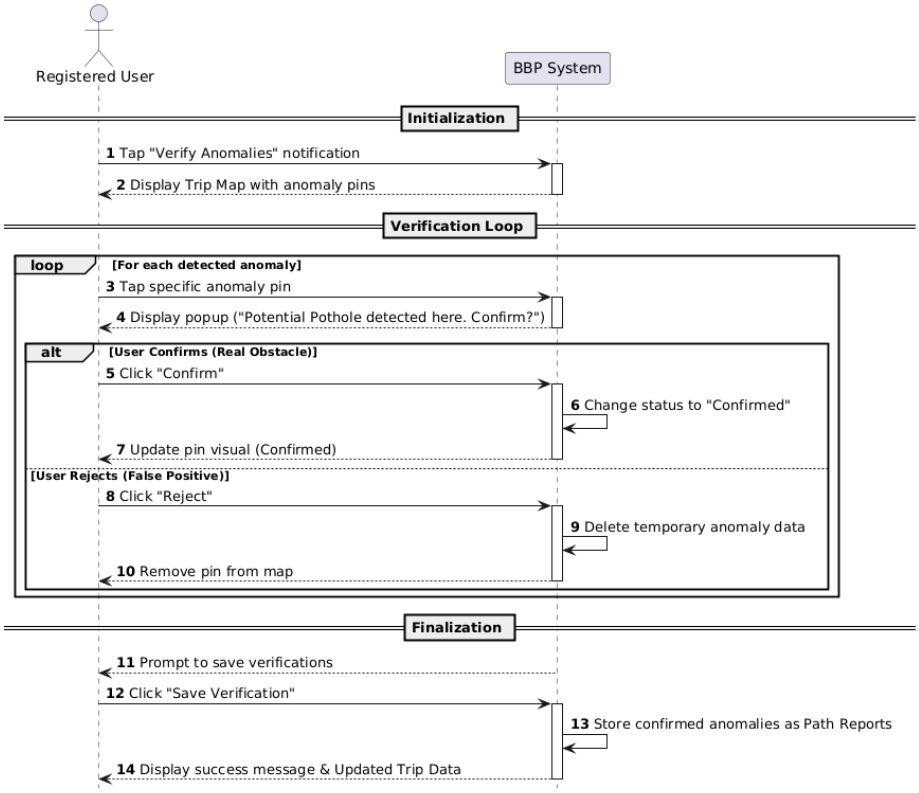


Figure 21: Verify Detected Data

ID	UC7
Name	Report Path Manually
Actor	Registered User
Entry conditions	The User has logged in and selected the "Report Path" function.
Input	Location selection, Status details
Events flow	1. The application displays the map. 2. The User navigates the map and taps to select a specific path segment or location. 3. The application displays a reporting form. 4. The User selects the Status (e.g., "Requires Maintenance") and Obstacle Type (e.g., "Flooding") from dropdown menus. 5. The User optionally adds a text note or photo. 6. The User sets the visibility toggle to "Public" (Publishable). 7. The User taps "Submit".
Exit conditions	• A new Path Report is created in the database. • If public, the data is immediately available to the community.
Output	Success message: "Report submitted successfully."
Exceptions	**Invalid Location:** If the selected point is not on a known bike path entity, the System warns the user: "No mapped bike path at this location."

Table 12: Use Case 7: Report Path Manually

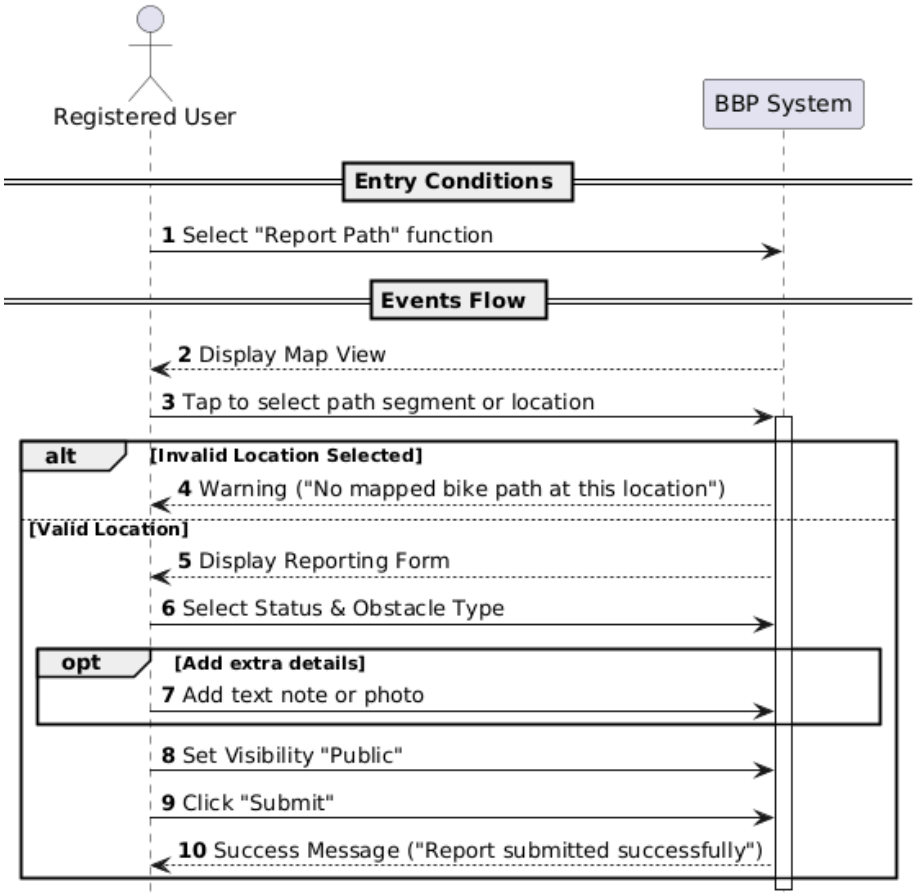


Figure 22: Report Path Manually

ID	UC8
Name	Publish Information
Actor	Registered User
Entry conditions	The User is viewing a private Trip or Report in their history.
Input	User command (Make Publishable)
Events flow	1. The User selects a past private trip or verified anomaly report. 2. The User taps the "Make Publishable" button. 3. The application displays a privacy warning: "This will share path conditions. Continue?" 4. The User confirms. 5. The System retrieves the data, strips personal identifiers (User ID linkage to the specific path segment data is anonymized). 6. The System updates the Public Inventory with the path quality/obstacle data. 7. The System triggers a recalculation of the Path Score for the affected segments.
Exit conditions	• The data is marked as "Public" in the user's view. • The community database is updated.
Output	Confirmation message; Updated Path Scores in the system.
Exceptions	**Server Error:** If the update fails, the System informs the user and retains the data state as Private.

Table 13: Use Case 8: Publish Information

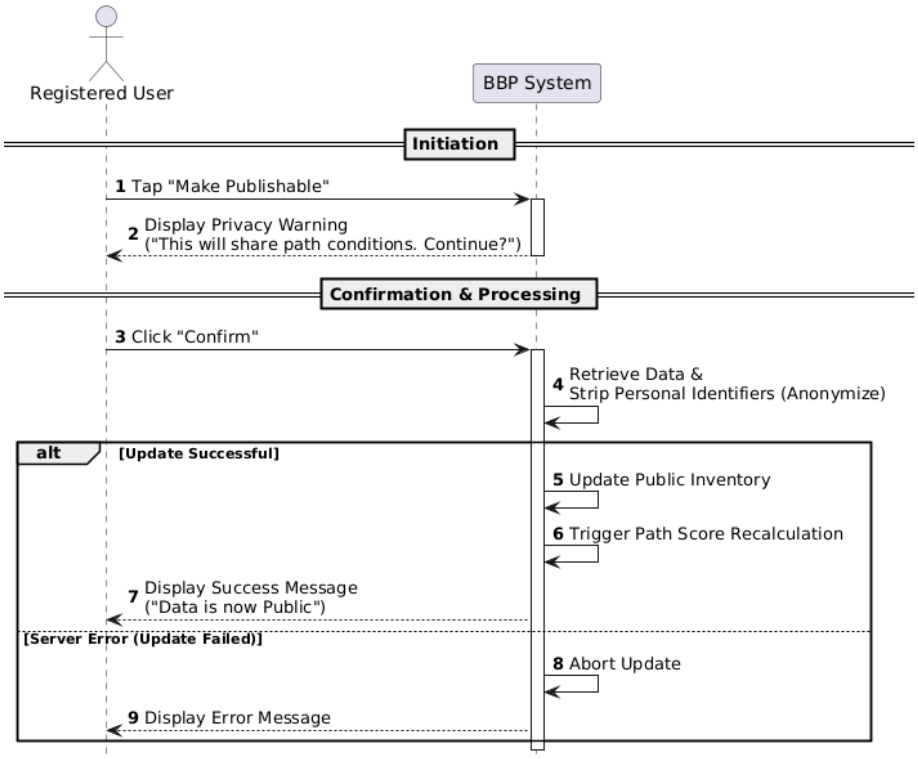


Figure 23: Publish Information

ID	UC9
Name	View Statistics and History
Actor	Registered User
Entry conditions	The User has logged in and is currently on the main dashboard.
Input	User navigation commands (Select Menu, Select Trip)
Events flow	1. The User navigates to the "My History" section from the main menu. 2. The System retrieves the chronological list of past recorded trips from the database. 3. The application displays the list of trips, showing summary metrics for each (e.g., Date, Total Distance, Duration). 4. The User scrolls through the list and taps on a specific trip card. 5. The System retrieves the full dataset for the selected trip, including GPS waypoints, speed telemetry, and associated weather data. 6. The application displays the detailed Trip View, which includes: • The route drawn on the map. • Graphs showing speed and elevation over time. • Weather conditions at the time of the ride.
Exit conditions	• The User has visualized the specific performance metrics of a past activity. • The User returns to the main list or dashboard.
Output	• Visual list of historical activities. • Detailed performance graphs and weather-enriched route map.
Exceptions	**No History Available:** If the User has not recorded any trips yet, the application displays a placeholder message: "No trips recorded yet. Start riding!" and offers a button to start a new recording.

Table 14: Use Case 9: View Statistics and History

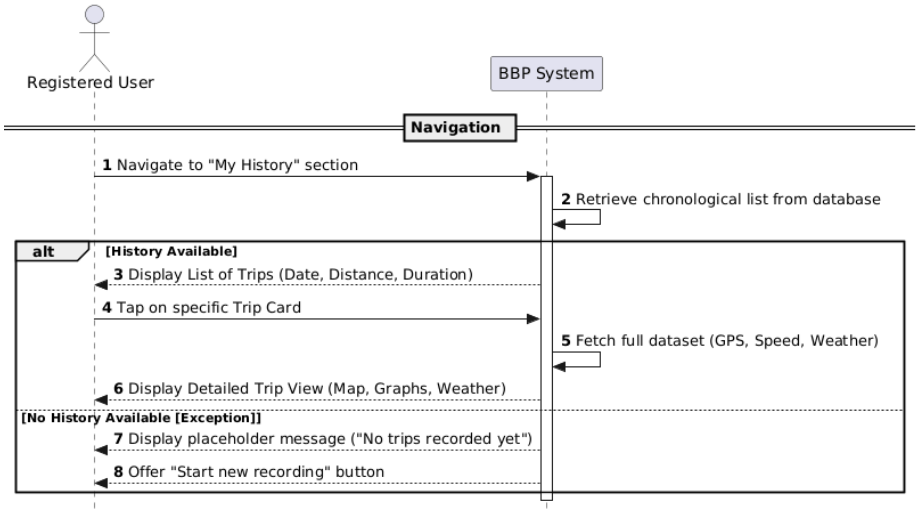


Figure 24: View Statistics And History

B.3 Requirements

This subsection summarizes the goals described in section A.1, mapping them to specific functional requirements and domain assumptions.

-[G1]: **Allow registered users to track their cycling activity**

- **[R1]:** The system allows Unregistered Users to sign-up providing personal details.
- **[R2]:** The system allows Registered Users to log-in to access their personal dashboard.
- **[R3]:** The system allows users to manually start and stop the recording of a trip via the application interface.
- **[R4]:** The system continuously tracks the user's geolocation (GPS coordinates) and time during the recording.
- **[D_ASS1]:** GPS Availability - It is assumed that the GPS signal is available and sufficiently accurate to allow for meaningful path reconstruction.

-[G2]: **Provide detailed statistics on trip performance**

- **[R2]:** The system allows Registered Users to log-in.
- **[R5]:** The system processes raw GPS data to calculate performance metrics, specifically total distance and average speed.
- **[R6]:** The system displays a summary of the statistics immediately after a trip is stopped.
- **[R7]:** The system archives the trip statistics in the user's personal history for future retrieval.

-[G3]: **Enrich trip data with contextual information**

- **[R8]:** The system connects to an external service to retrieve meteorological data (temperature, wind, condition) based on the trip's timestamp and location.
- **[R9]:** The system merges the retrieved weather data with the trip statistics.
- **[DEP2]:** External Weather Service - The system depends on the availability of an external API to fetch real-time and historical meteorological data.
- **[DEP3]:** Mobile Data Connectivity - The system depends on the mobile network to fetch weather data.

-[G4]: **Enable manual reporting of path status**

- **[R2]:** The system allows Registered Users to log-in.

- **[R10]:** The system allows users to select a specific location or segment on the map.
- **[R11]:** The system allows users to input status details (e.g., "Requires Maintenance") and obstacle types.
- **[R12]:** The system saves the manual report to the user's personal records.
- **[D_ASS5]:** User Integrity - It is assumed that the majority of users provide truthful manual reports regarding path conditions.

-[G5]: **Automate data collection via mobile sensors**

- **[R13]:** The system monitors device sensors (accelerometer and gyroscope) in the background while the user is cycling.
- **[R14]:** The system identifies potential road anomalies (e.g., potholes) based on vertical jolts and orientation changes.
- **[R15]:** The system logs the anomaly type and location internally for later verification.
- **[D_ASS2]:** Device Sensor Capability - It is assumed that users' smartphones are equipped with functioning accelerometers and gyroscopes.
- **[D_ASS3]:** Device Placement - It is assumed the phone is placed in a position (e.g., pocket, handlebar) that allows perceiving vibrations.

-[G6]: **Ensure accuracy of automatically detected data**

- **[R16]:** The system notifies the user after a trip if potential anomalies were detected.
- **[R17]:** The system displays a map or list of detected anomalies for verification.
- **[R18]:** The system allows the user to explicitly "Confirm" or "Reject" (False Positive) a detected anomaly.
- **[R19]:** The system discards data marked as "Reject" and stores data marked as "Confirm" as valid path information.
- **[D_ASS5]:** User Integrity - It is assumed that users generally provide truthful confirmations.

-[G7]: **Enable the publication of collected information**

- **[R2]:** The system allows Registered Users to log-in.
- **[R20]:** The system allows users to select validated data (manual or automated) from their history.
- **[R21]:** The system allows users to mark specific data as "Publishable".

- **[R22]:** The system updates the public inventory with the published path quality data.
- **[CON1]:** Regulatory & Privacy - Private trips must remain inaccessible unless explicitly published.

-[G8]: **Support route planning for all users**

- **[R23]:** The system allows both Registered and Unregistered users to access the "Explore" section.
- **[R24]:** The system allows users to input an Origin and a Destination.
- **[R25]:** The system calculates available routes between the two points.
- **[DEP1]:** External Map Service - The system depends on a third-party Map Provider for road network topology.

-[G9]: **Evaluate and rank available paths**

- **[R25]:** The system calculates available routes.
- **[R26]:** The system computes a "Path Score" for each route based on path status (from the public inventory) and efficiency.
- **[R27]:** The system visualizes the routes on the map, highlighting the highest-scored path.
- **[R28]:** The system displays a summary of the route details (distance, estimated time, score).

-[G10]: **Detect cycling activity automatically**

- **[R29]:** The system periodically monitors GPS speed in the background.
- **[R30]:** The system automatically triggers the "Start Recording" function when the speed exceeds a specific cycling threshold (10 km/h) for a sustained period (30 seconds).
- **[R31]:** The system automatically stops the recording when the user stops moving for a defined timeframe (5 minutes).
- **[D_ASS4]:** Distinguishability of Speed - It is assumed that cycling speeds are sufficiently distinct from walking and driving to allow automated inference.

-[G11]: **Protect user privacy while enabling collaboration**

- **[R32]:** The system stores all recorded trips as "Private" by default.
- **[R33]:** The system anonymizes personal identifiers before transferring data to the public inventory during the publication process.
- **[CON1]:** Regulatory & Privacy - The system must ensure no personal trip data is exposed unless explicitly published.

B.4 Traceability Matrix

The following table maps the Use Cases described in Section 3.2.2 to the specific Requirements defined in Section 3.2.3.

Use Case	Requirements
UC1: Sign Up	[R1]
UC2: Log In	[R2]
UC3: Plan Route	[R23], [R24], [R25], [R26], [R27], [R28]
UC4: Record Personal Trip	[R2], [R3], [R4], [R5], [R6], [R7], [R8], [R9]
UC5: Automated Data Collection	[R2], [R13], [R14], [R15], [R29], [R30], [R31]
UC6: Verify Detected Data	[R2], [R16], [R17], [R18], [R19]
UC7: Report Path Manually	[R2], [R10], [R11], [R12]
UC8: Publish Information	[R2], [R20], [R21], [R22], [R33]
UC9: View Statistics & History	[R2], [R7]

Table 15: Traceability Matrix: Use Cases vs Requirements (Updated)

C Performance Requirements

The Best Bike Paths (BBP) system must ensure fluid interaction and accurate tracking, balancing data precision with battery efficiency.

- **[PR1] Response Time:** The mobile application shall maintain a high responsiveness. Interactions such as starting/stopping a recording or navigating menus should have a latency of less than **1 second**.
- **[PR2] Route Calculation Latency:** When a user requests a route (Scenario 7), the system should retrieve and visualize the calculated paths within **3 seconds** under normal network conditions.
- **[PR3] Sensor Data Frequency:** To accurately detect anomalies (potholes/bumps) via the "Automated Mode" (Scenario 3), the application must poll the accelerometer and gyroscope at a minimum frequency of **50 Hz** while the vehicle is in motion.
- **[PR4] Location Tracking Accuracy:** During trip recording, GPS coordinates must be sampled at a minimum rate of **1 point every 5 seconds** or **every 10 meters**, with a horizontal accuracy within 10 meters.
- **[PR5] Concurrent Users:** The backend system shall support at least **1,000 concurrent users** sending telemetry data or requesting routes without service degradation.
- **[PR6] Offline Buffering:** The mobile application must be capable of buffering at least **24 hours** of trip data and sensor logs locally in case of connectivity loss.

- **[PR7] Battery Consumption:** The background activity detection and recording service must be optimized to consume no more than **10% of the device battery per hour** of active tracking.

D Design Constraints

D.1 Standards compliance

- **[DC1] GDPR Compliance:** In accordance with **[CON1]**, all personal data (trip history, precise geolocation) must be stored in compliance with the General Data Protection Regulation (GDPR). The system must allow users to export or delete their data upon request.
- **[DC2] Map Standards:** The system must utilize standard geospatial data formats (e.g., GeoJSON) for storing and exchanging path and anomaly data.
- **[DC3] Communication Security:** All data transmission between the mobile client and the backend server must be encrypted.

D.2 Hardware limitations

- **[DC4] Smartphone Sensors:** The application requires the user's device to be equipped with a functioning **GPS/GNSS receiver**, **accelerometer**, and **gyroscope**. The system cannot function fully on devices lacking these sensors.
- **[DC5] Storage Limits:** To respect device storage limitations, the local buffer for offline trips (DC3) shall utilize a circular buffer or a compression mechanism, capping local storage usage at **500 MB**.
- **[DC6] Battery Consumption:** The "Automated Mode" and background tracking must be optimized to not drain more than **10% of the device's battery per hour** of active usage.
- **[DC7] Device Placement:** As noted in Domain Assumption **[D_ASS3]**, the accuracy of pothole detection is constrained by the physical placement of the phone.

D.3 Any other constraint

- **[DC8] OS Compatibility:** The mobile application must be compatible with **Android 10.0**+ and **iOS 14.0**+ to ensure access to modern background processing APIs required for Requirement R7.
- **[DC9] Network Dependency for Enrichment:** The weather enrichment feature (Scenario 2) and route planning (Scenario 7) are strictly dependent on an active internet connection. These features are disabled in offline mode.

E Software System Attributes

E.1 Reliability

- **[SSA1] Data Persistence & Integrity:** The system must ensure **zero data loss** for recorded trips. In the event of an application crash or unexpected shutdown (e.g., battery depletion), the current trip state must be recoverable from the local storage upon the next launch.
- **[SSA2] Detection Accuracy:** The automated anomaly detection algorithm should aim to minimize user fatigue by maintaining a False Positive rate below **20%** (i.e., at least 80% of detected anomalies should be real road defects).

E.2 Availability

- **[SSA3] Offline Resilience:** The core tracking functionality (recording trips and collecting sensor data) must be **100% available** regardless of network status. Synchronization with the server shall occur transparently when connectivity is restored.
- **[SSA4] Server Uptime:** The BBP Backend Server must guarantee **99.9% uptime** during peak usage hours (06:00 - 22:00), ensuring users can always log in and sync their history.

E.3 Security

- **[SSA5] Data Encryption:** All sensitive data transmission between the mobile client and the backend server (authentication, geolocation history) must be encrypted via HTTPS.
- **[SSA6] Privacy by Default:** All user trips are "Private" by default. The system must ensure that no personal trip data is exposed to the public map unless the user explicitly executes the "Publish Information" use case **[UC7]**.
- **[SSA7] Authentication:** Access to personal history and reporting features requires secure login (Token-based authentication, e.g., **JWT**) to prevent session hijacking.

E.4 Maintainability

- **[SSA8] Modular Architecture:** The "Anomaly Detection Module" must be decoupled from the main application logic. This allows the development team to update or tune the detection algorithm (e.g., tweaking sensitivity thresholds) without requiring a complete overhaul of the mobile application.

- **[SSA9] Code Documentation:** The source code must follow standard documentation practices (e.g., Javadoc for Java/Android, SwiftDoc for iOS) to facilitate future handover to other developers in the association.

E.5 Portability

- **[SSA10] Cross-Platform Consistency:** The mobile application must provide feature parity between the **Android** and **iOS** versions, ensuring that the sensor data collection behaves consistently across different operating systems.
- **[SSA11] Backend Deployment:** The server-side components should be containerized (e.g., using Docker) to allow deployment on any standard cloud infrastructure (AWS, Azure, or on-premise servers) without environment-specific configurations.

4 Formal Analysis

A Alloy code

```
-- SIGNATURES

-- Basic types
sig Location {}

-- Enumerations for path status
abstract sig PathStatus {}
one sig Optimal extends PathStatus {}
one sig Medium extends PathStatus {}
one sig RequiresMaintenance extends PathStatus {}

-- Enumerations for anomaly types
abstract sig AnomalyType {}
one sig Pothole extends AnomalyType {}
one sig RoughSurface extends AnomalyType {}
one sig Obstacle extends AnomalyType {}

-- Enumerations for verification status
abstract sig VerificationStatus {}
one sig Pending extends VerificationStatus {}
one sig Confirmed extends VerificationStatus {}
one sig Rejected extends VerificationStatus {}

-- Enumerations for visibility
abstract sig Visibility {}
one sig PrivateVis extends Visibility {}
one sig PublicVis extends Visibility {}

-- Boolean type
abstract sig Bool {}
one sig True extends Bool {}
one sig False extends Bool {}

-- Users
sig RegisteredUser {
    trips : set Trip,
    manualReports : set PathReport
}

-- Physical path segment in the world
sig PathSegment {
    location : one Location,
    publishedReports : set PathReport
}

-- Trip recorded by a user
sig Trip {
    owner : one RegisteredUser,
    path : set PathSegment,
    detectedAnomalies : set SensorAnomaly,
    visibility : one Visibility
}

-- Automated sensor data
sig SensorAnomaly {
    tripContext : one Trip,
    anomalyType : one AnomalyType,
    anomalyLocation : one Location,
    verification : one VerificationStatus,
    resultingReport : lone PathReport
}
```

```
-- Validated path information
sig PathReport {
    reporter : one RegisteredUser,
    segment : one PathSegment,
    statusReported : one PathStatus,
    isAutomated : one Bool,
    visibility : one Visibility
}

-- Route visualization
sig Route {
    segments : some PathSegment
}


-- FACTS

-- F1: Trip Ownership
fact TripOwnership {
    all t: Trip | t in t.owner.trips
    all u: RegisteredUser, t: Trip | t in u.trips iff t.owner = u
}

-- F2: Anomaly Context
fact AnomalyBelongsToTrip {
    all a: SensorAnomaly | a in a.tripContext.detectedAnomalies
}

-- F3: Report Authorship
fact ReportAuthorship {
    all r: PathReport | {
        (r.isAutomated = True implies
            (one a: SensorAnomaly |
                a.resultingReport = r and
                a.tripContext.owner = r.reporter))
        and
        (r.isAutomated = False implies r in r.reporter.manualReports)
    }
}

-- F4: Verification Logic
fact VerificationLogic {
    all a: SensorAnomaly | {
        (a.verification = Confirmed iff one r: PathReport | r = a.
            ↪ resultingReport)
        and
        ((a.verification = Pending or a.verification = Rejected)
            implies no a.resultingReport)
        and
        (some a.resultingReport implies
            a.resultingReport.reporter = a.tripContext.owner)
    }
}

-- F5: Location Consistency
fact LocationConsistency {
    all a: SensorAnomaly, r: PathReport |
        a.resultingReport = r implies r.segment.location = a.anomalyLocation
}

-- F6: Trip Geography
fact AnomalyOnPath {
    all a: SensorAnomaly |
        some s: PathSegment |
            s in a.tripContext.path and
            s.location = a.anomalyLocation
}
```

```
-- F7: Privacy by Default
fact PrivacyByDefault {
    all t: Trip | (t.visibility = PrivateVis or t.visibility = PublicVis)
}

-- F8: Privacy Inheritance for Automated Reports
fact PrivacyInheritance {
    all a: SensorAnomaly, r: PathReport |
        (a.resultingReport = r and a.tripContext.visibility = PrivateVis)
        implies r.visibility = PrivateVis
}

-- F9: Published Reports Visibility
fact PublishedReportsArePublic {
    all s: PathSegment, r: PathReport |
        r in s.publishedReports implies r.visibility = PublicVis
}

-- F10: Manual Report Ownership
fact ManualReportOwnership {
    all r: PathReport |
        r.isAutomated = False implies r in r.reporter.manualReports
}

-- F11: No Duplicate Reports
fact NoDuplicateReports {
    all u: RegisteredUser, s: PathSegment |
        lone r: PathReport |
            r.reporter = u and
            r.segment = s and
            r in s.publishedReports
}

-- F12: Routes Use Segments with Data
fact RoutesHavePublishedData {
    all r: Route, s: r.segments |
        #(s.publishedReports) > 0
}

-- F13: All locations connected
fact AllLocationsConnected {
    all loc: Location | one s: PathSegment | loc = s.location
}

-- F14: All reports connected
fact AllReportsConnected {
    all r: PathReport | {
        (r.isAutomated = True implies one a: SensorAnomaly | a.
            ↪ resultingReport = r)
        or
        (r.isAutomated = False implies one u: RegisteredUser | r in u.
            ↪ manualReports)
    }
}

-- F15: All anomalies connected
fact AllAnomaliesConnected {
    all a: SensorAnomaly | one t: Trip | a in t.detectedAnomalies
}


-- ASSERTIONS - VERIFY CORE REQUIREMENTS

-- A1: No rejected anomalies create reports
assert NoRejectedReports {
    all a: SensorAnomaly |
        a.verification = Rejected implies no a.resultingReport
}
```

```
-- A2: All automated reports come from confirmed anomalies
assert AutomatedReportsAreVerified {
    all r: PathReport |
        r.isAutomated = True implies
        (one a: SensorAnomaly |
            a.resultingReport = r and
            a.verification = Confirmed)
}

-- A3: Privacy guarantee
assert PrivacyGuarantee {
    all t: Trip |
        t.visibility = PrivateVis implies
        (all a: t.detectedAnomalies |
            (some a.resultingReport) implies
            a.resultingReport.visibility = PrivateVis)
}

-- A4: Published reports are only public
assert NoPrivateInPublishedInventory {
    all s: PathSegment, r: s.publishedReports |
        r.visibility = PublicVis
}

-- A5: Every report has an author
assert AllReportsHaveAuthor {
    all r: PathReport | one u: RegisteredUser | r.reporter = u
}

-- A6: Anomalies are only on traveled paths
assert AnomaliesOnTraveledPaths {
    all a: SensorAnomaly |
        some s: a.tripContext.path | s.location = a.anomalyLocation
}

-- A7: No orphan trips
assert NoOrphanTrips {
    all t: Trip | one u: RegisteredUser | t.owner = u
}

-- A8: Pending anomalies await user action
assert PendingAnomaliesNoReports {
    all a: SensorAnomaly |
        a.verification = Pending implies no a.resultingReport
}

-- Checks

check NoRejectedReports for 5
check AutomatedReportsAreVerified for 5
check PrivacyGuarantee for 5
check NoPrivateInPublishedInventory for 5
check AllReportsHaveAuthor for 5
check AnomaliesOnTraveledPaths for 5
check NoOrphanTrips for 5
check PendingAnomaliesNoReports for 5


-- PREDICATES

-- S1: Manual insertion of path information
pred Scenario_ManualPathReporting {
    some u: RegisteredUser, s: PathSegment, r: PathReport | {
        r in u.manualReports
        r.reporter = u
        r.segment = s
        r.isAutomated = False
```

```
        r.statusReported = RequiresMaintenance
        r.visibility = PublicVis
        r in s.publishedReports
    }
}

-- S2: Automated detection with confirmation
pred Scenario_AutomatedDetectionConfirmed {
    some u: RegisteredUser, t: Trip, a: SensorAnomaly, s: PathSegment, r:
        ↪ PathReport | {
        t.owner = u
        s in t.path
        a in t.detectedAnomalies
        a.tripContext = t
        a.anomalyLocation = s.location
        a.anomalyType = Pothole
        a.verification = Confirmed
        a.resultingReport = r
        r.reporter = u
        r.segment = s
        r.isAutomated = True
        r.visibility = PublicVis
        r in s.publishedReports
    }
}

-- S3: False positive rejection (CORE for Group 2)
pred Scenario_FalsePositiveRejected {
    some u: RegisteredUser, t: Trip, a: SensorAnomaly | {
        t.owner = u
        a in t.detectedAnomalies
        a.tripContext = t
        a.anomalyType = Pothole
        a.verification = Rejected
        no a.resultingReport
        no r: PathReport |
            r.isAutomated = True and
            r.reporter = u and
            r.segment.location = a.anomalyLocation
    }
}

-- S4: Private trip with pending anomalies
pred Scenario_PrivateTripWithPendingAnomalies {
    some u: RegisteredUser, t: Trip, a: SensorAnomaly | {
        t.owner = u
        t.visibility = PrivateVis
        a in t.detectedAnomalies
        a.tripContext = t
        a.verification = Pending
        no a.resultingReport
        no r: PathReport | r.reporter = u and r in PathSegment.
            ↪ publishedReports
    }
}



-- S5: User publishes previously private data
pred Scenario_PublishPrivateData {
    some u: RegisteredUser, t: Trip, a: SensorAnomaly, r: PathReport, s:
        ↪ PathSegment | {
        t.owner = u
        t.visibility = PrivateVis
        a in t.detectedAnomalies
        a.verification = Confirmed
        a.resultingReport = r
        r.visibility = PublicVis
```

```
        r.segment = s
        r in s.publishedReports
        t.visibility = PrivateVis
    }
}

-- S6: Route visualization
pred Scenario_RouteVisualization {
    some route: Route | {
        #(route.segments) ≥ 2
        all s: route.segments |
            #(s.publishedReports) > 0
        #(route.segments.publishedReports.reporter) ≥ 1
    }
}

-- S7: Multiple users contribute data
pred Scenario_MultipleContributors {
    some s: PathSegment, r1, r2: PathReport, u1, u2: RegisteredUser | {
        u1 ≠ u2
        r1.segment = s and r2.segment = s
        r1.reporter = u1 and r2.reporter = u2
        r1 in s.publishedReports
        r2 in s.publishedReports
        r1.statusReported ≠ r2.statusReported
    }
}


-- WORLDS

--  Used to not show orphans PathSegments in worlds
pred CleanWorld {
    all s: PathSegment | (some t: Trip | s in t.path) or (some r: PathReport
        ↪ | r.segment = s)
}

-- World showing the confirmation workflow
pred World_ConfirmationWorkflow {
    some u: RegisteredUser, t: Trip | {
        t.owner = u
        #(t.detectedAnomalies) = 3
        one a1: t.detectedAnomalies | a1.verification = Confirmed
        one a2: t.detectedAnomalies | a2.verification = Rejected
        one a3: t.detectedAnomalies | a3.verification = Pending
        #{a: t.detectedAnomalies | some a.resultingReport} = 1
    }
}

-- World 1: Simple registration (initial state)
pred World1 {
    #RegisteredUser = 2
    #Trip = 0
    #PathReport = 0
    #SensorAnomaly = 0
}

-- World 2: Manual reporting only
pred World2 {
    #RegisteredUser = 2
    #Trip = 2
    #PathSegment = 3
    #PathReport > 1
    #SensorAnomaly = 0
    // All reports are manual
    all r: PathReport | r.isAutomated = False
}
```

```
-- World 3: Automated detection with confirmation
pred World3 {
    #RegisteredUser = 1
    #Trip = 1
    #SensorAnomaly = 2
    #PathReport > 0
    // At least one anomaly confirmed
    some a: SensorAnomaly | a.verification = Confirmed
}

-- World 4: Mixed manual and automated reports
pred World4_MixedReports {
    some u: RegisteredUser, t: Trip, s1, s2: PathSegment, r1, r2: PathReport
        ↪ | {
        t.owner = u
        s1 ≠ s2

        // Manual report
        r1 in u.manualReports
        r1.isAutomated = False
        r1.segment = s1
        r1.visibility = PublicVis
        r1 in s1.publishedReports

        // Automated report from confirmed anomaly
        some a: t.detectedAnomalies | {
            a.verification = Confirmed
            a.resultingReport = r2
            r2.segment = s2
            r2.isAutomated = True
            r2.visibility = PublicVis
            r2 in s2.publishedReports
        }
    }
}

-- World 5: Privacy demonstration
pred World5_PrivateVsPublic {
    some u: RegisteredUser, t1, t2: Trip | {
        t1.owner = u
        t2.owner = u
        t1 ≠ t2

        // Private trip with pending anomalies
        t1.visibility = PrivateVis
        some a1: t1.detectedAnomalies | a1.verification = Pending

        // Public trip with confirmed and published data
        t2.visibility = PublicVis
        some a2: t2.detectedAnomalies | {
            a2.verification = Confirmed
            some a2.resultingReport
            a2.resultingReport.visibility = PublicVis
        }
    }
}

-- Execution Commands
run World1_Clean { World1 and CleanWorld } for 5
run World2_Clean { World2 and CleanWorld } for 5
run World3_Clean { World3 and CleanWorld } for 5
run World4_Clean { World4_MixedReports and CleanWorld } for 5
run World5_Clean { World5_PrivateVsPublic and CleanWorld } for 5
run Workflow_Clean { World_ConfirmationWorkflow and CleanWorld } for 5
```

A.1 Worlds

The following instances (Worlds) have been generated to visually validate the consistency of the model against specific functional scenarios. Each world represents a snapshot of the system state satisfying a distinct set of constraints.

World 1: Initial System State This instance represents the baseline state of the BBP system. It depicts the existence of registered users prior to the creation of any trips, anomalies, or reports. This world confirms that the static structural constraints of the domain model are consistent even in the absence of dynamic activity.



Figure 25: World 1: Initial State with registered users but no activity.

World 2: Manual Reporting Workflow This scenario illustrates the system's behavior when relying exclusively on user inputs. It depicts a state containing multiple trips and path reports, where all reports are explicitly marked as manual (isAutomated = False). This world validates that the system can successfully manage crowdsourced data without requiring the intervention of the automated sensor subsystem.

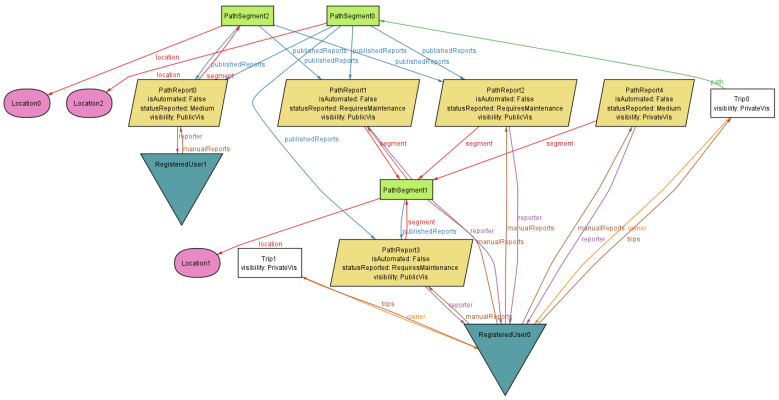


Figure 26: World 2: Scenario focusing on Manual Reporting only.

World 3: Automated Detection and Verification This instance focuses on the lifecycle of automatically detected data. It shows the generation of SensorAnomaly entities linked to a trip and verifies the transition logic where a user confirms an anomaly, resulting in the creation of a valid PathReport. This world is essential to verify that the automated collection and user verification loop functions correctly.

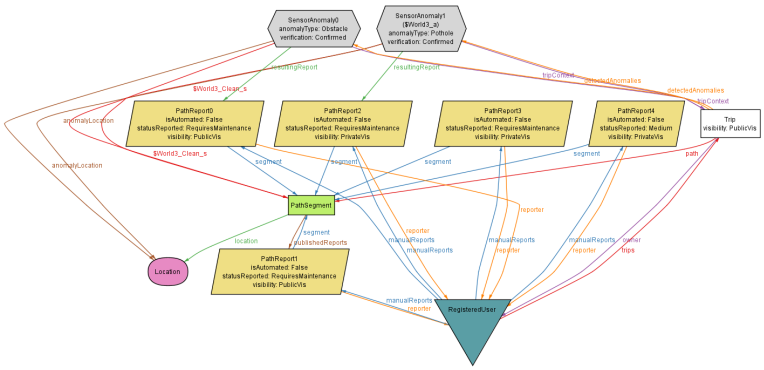


Figure 27: World 3: Scenario showing Automated Detection and Verification.

World 4: Aggregation of Heterogeneous Data This world demonstrates the system's capability to integrate different data sources simultaneously. It presents a complex state where manual reports coexist with automated reports derived from confirmed sensor anomalies. This validation ensures that the PathReport entity correctly abstracts the data origin (manual vs. automated) while maintaining consistency in the public inventory.

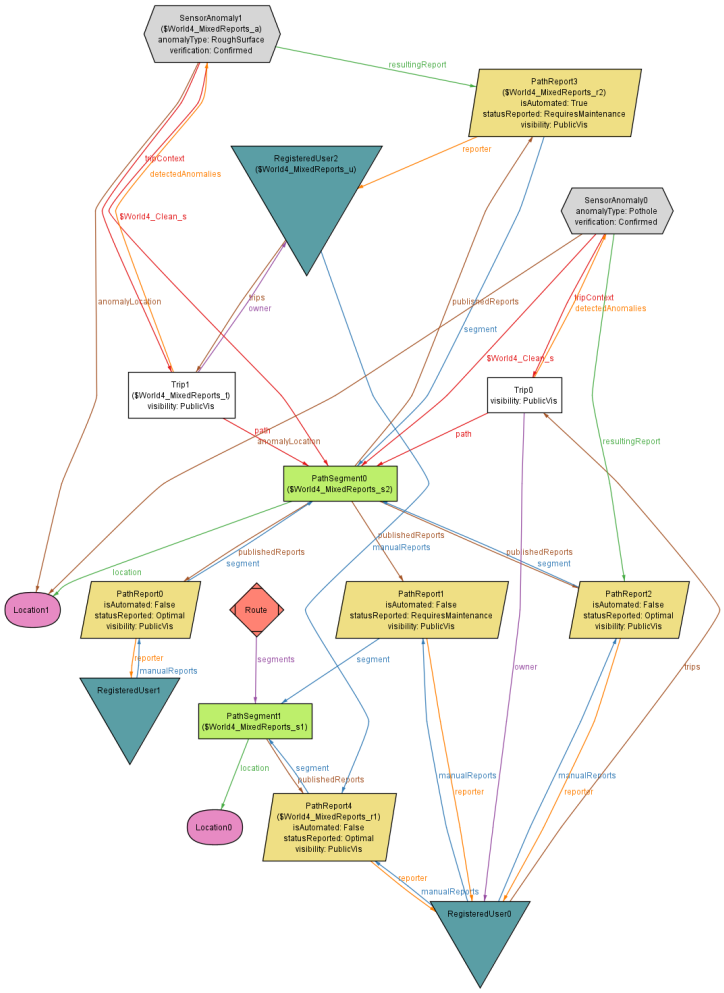


Figure 28: World 4: Coexistence of Manual and Automated reports.

World 5: Privacy Constraints This scenario validates the "Privacy by Default" requirement and the explicit publication mechanism. It contrasts two distinct trips owned by the same user: a *Private* trip, where detected anomalies remain pending and isolated, and a *Public* trip, where confirmed data is visible to the community. This world proves that the model correctly enforces visibility boundaries, preventing the accidental leakage of private user data.

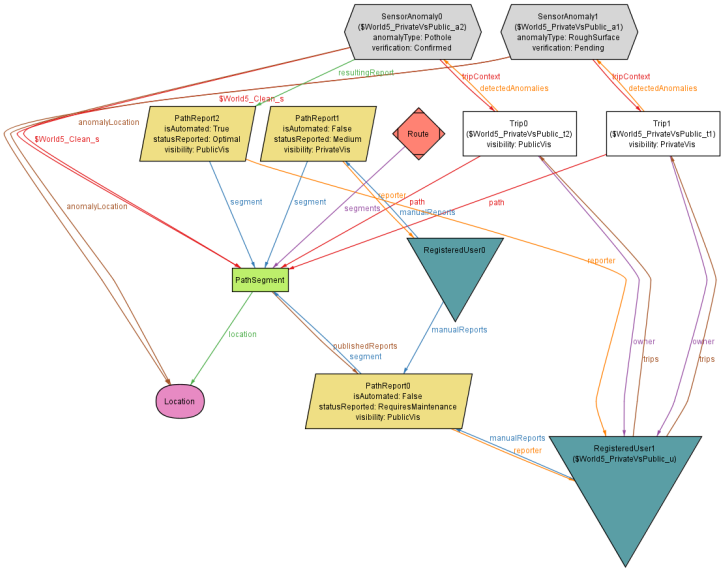


Figure 29: World 5: Privacy mechanisms and data publication.

Anomaly Verification Workflow This specific instance validates the lifecycle of sensor data, showcasing the three possible states of an anomaly within a single trip. It depicts a scenario where multiple anomalies coexist: one confirmed (generating a report), one rejected (discarded), and one pending (awaiting action). This visualizes the correctness of the state machine logic formalized in Fact F4, ensuring that only verified data impacts the public inventory.

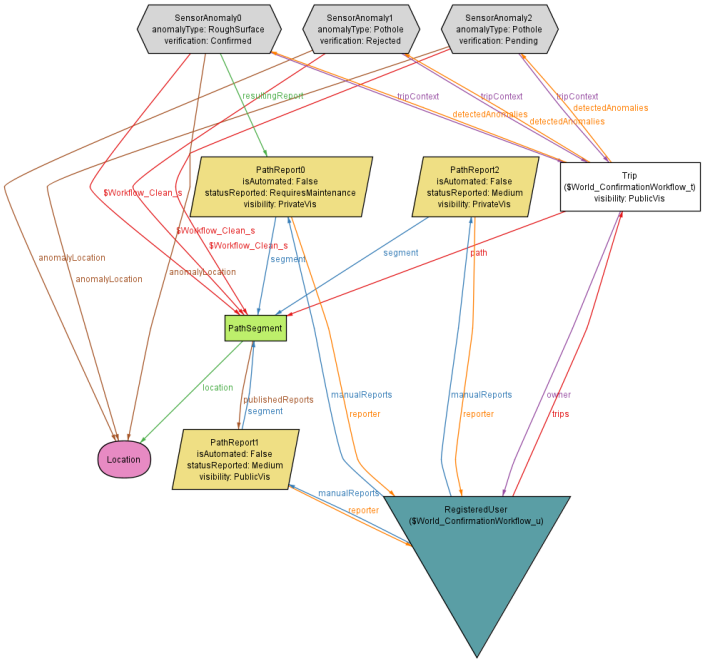


Figure 30: Verification Workflow: Handling Pending, Confirmed, and Rejected anomalies.

5 Effort spent

Student	Time for S.1	Time for S.2	Time for S.3	Time for S.4
Davide Martini	8h	13h	18h	4h
Filippo Maretti	6h	10h	10h	18h